

“The Agent Failed” Is Not an Operational Signal: A Reference for Evals, Observability and Recovery

What a production agent runtime must observe, how it turns that evidence into a failure class and an execution certainty, which recovery it may then choose, and how evals verify that the recovery was correct. With a recorded POC of 25 injected faults through three recovery layers.

R

R1 + R2 · Production AI Engineering

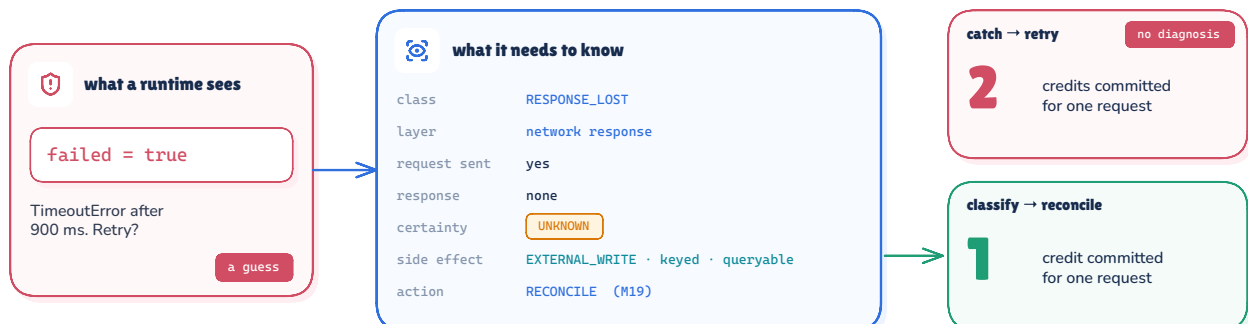
64 min read · Published 2026-10-07 · Technical deep dive

— PRODUCTION AI ENGINEERING · R1 + R2 · EVALS, OBSERVABILITY & RELIABILITY

“The agent failed” is not an operational signal.

a timeout is not a failure.
it is missing information.

Observability creates evidence. Classification turns it into state.
Reliability chooses the next safe action. Evals verify it.



same fault · same tools · same model · only the recovery layer differs

run 2026-10-07-recorded · S09

KEY ■ unknown ■ diagnosis ■ evidence ■ a guess · duplicate ■ verified

The reference companion to the Medium edition. Every number is substituted from the recorded run; every external claim links to a numbered source that was fetched and quoted in `research/sources.md`.

About this edition

The Medium edition makes one argument: **a production agent runtime cannot recover safely from “the agent failed”**. It has to know what failed, whether the far side executed, what the tool’s side effects are, and what state survived, and only then choose an action. This edition is the reference behind that argument, for platform engineers, SREs and architects who build or operate agents that change things in the world. It covers:

- a failure taxonomy precise enough that each class has its own recovery;
- **execution certainty** (`NOT_EXECUTED` , `EXECUTED` , `UNKNOWN`) as a first-class state, and the evidence rules that establish it;
- a deterministic **recovery matrix** that maps class, certainty, the tool’s side-effect contract and persisted state to one of nine actions;
- the telemetry and checkpoint data that recovery actually needs, and what it does not;
- evals in five families, including **recovery evals**, and how they gate releases;
- a recorded POC: 25 preregistered faults through 3 recovery layers, 8 mutants of the recovery layer, a deterministic model change, 96 real-model calls in a release-gate slice, and the whole fault set again end to end with a real model deciding (78 calls).

HOW TO READ THE FIGURES

- requests, the edge of the system ■ orchestration, the workflow store, a human
- the agent and its model: probabilistic ■ context and retrieval ■ tools and providers' APIs
- deterministic: gates, classifier, recovery matrix ■ control plane: frozen policy, release gate
- evidence ■ enterprise systems (simulated) ■ failure, deny, a duplicate effect
- executed, verified

ARCHITECTURE conceptual design **MEASURED** a recorded POC result

RECORDED one recorded scenario run **SIMULATED** a simulated external system

Statements are marked by what they rest on:

- **Sourced.** An established concept with a numbered reference, such as gRPC’s warning that a deadline can expire after the operation succeeded ^[2]. Every source is quoted in `research/sources.md` together with what it does *not* support.
- **Our synthesis.** A position this series takes. *Execution certainty*, *recovery evals* and the *recovery matrix* are in this category: no standard defines them, and Jepsen’s `:ok` / `:fail` / `:info`

outcomes ^[1] are the closest prior art, an analogy rather than a source.

- **Implemented / measured / recorded.** Behaviour of this article's POC. Numbers are substituted at build time from `recovery_poc/runs/2026-10-07-recorded/facts.json`. Results from earlier articles are labelled **historical** and are read from those packages' own recorded facts.

1 • Executive summary

An agent run that ends in an exception has, from the runtime's point of view, one bit of information: something did not return. The production questions that matter are different. Which layer failed? Was the request even sent? Did the provider execute it? Did an external effect happen? What did the workflow persist? Is it safe to do the same thing again? The answers decide whether the correct next step is to retry, resume, reconcile, continue, fall back, compensate, escalate or abort, and those actions are not interchangeable. A retry after a lost response can pay a customer twice. Aborting after a lost response can tell a customer their credit failed when it committed.

The architecture in this edition separates three disciplines that are usually blurred:

- **Observability** records operational facts at every boundary: `request_sent`, `response_received`, transport outcome, status, operation id, journal state.
- **Reliability** is a deterministic policy that turns those facts into a failure class, an execution certainty and an action.
- **Evals** check that the behaviour, *and the recovery*, are still correct: before release, and on live traffic.

The POC injects 25 faults covering every layer of the failure taxonomy into one support-agent workflow (retrieve, look up charges, decide, validate, authorize, credit, open a ticket, notify) and runs each through three runtimes that differ only in their recovery layer:

	A0 naive (catch → retry)	A1 idempotent retry	A2 classified
scenarios with a duplicate external effect	10	5	0
correct outcome (status and effects equal the preregistered oracle; a required escalation counts)	10 of 25	14 of 25	25 of 25
final answers contradicting the ledgers	1	1	0
retries of refusals no retry can change	8	8	0
failure events with a class and a certainty	0 of 32	0 of 32	38 of 38
reconciliation queries (the cost)	0	0	12

A2's recorded decisions matched the preregistered oracle in 25 of 25 scenarios. Each of the 8 preregistered mutants of A2's recovery layer was caught by at least one eval check. Two local models, asked to make the same decision the scripted model makes, both failed the preregistered release gate, and none of their proposals that should not have executed got past the runtime's gates. Run end to end with qwen3:8b making every decision, the 25 faults gave the same result: 10, 5 and 0 scenarios with a duplicate effect.

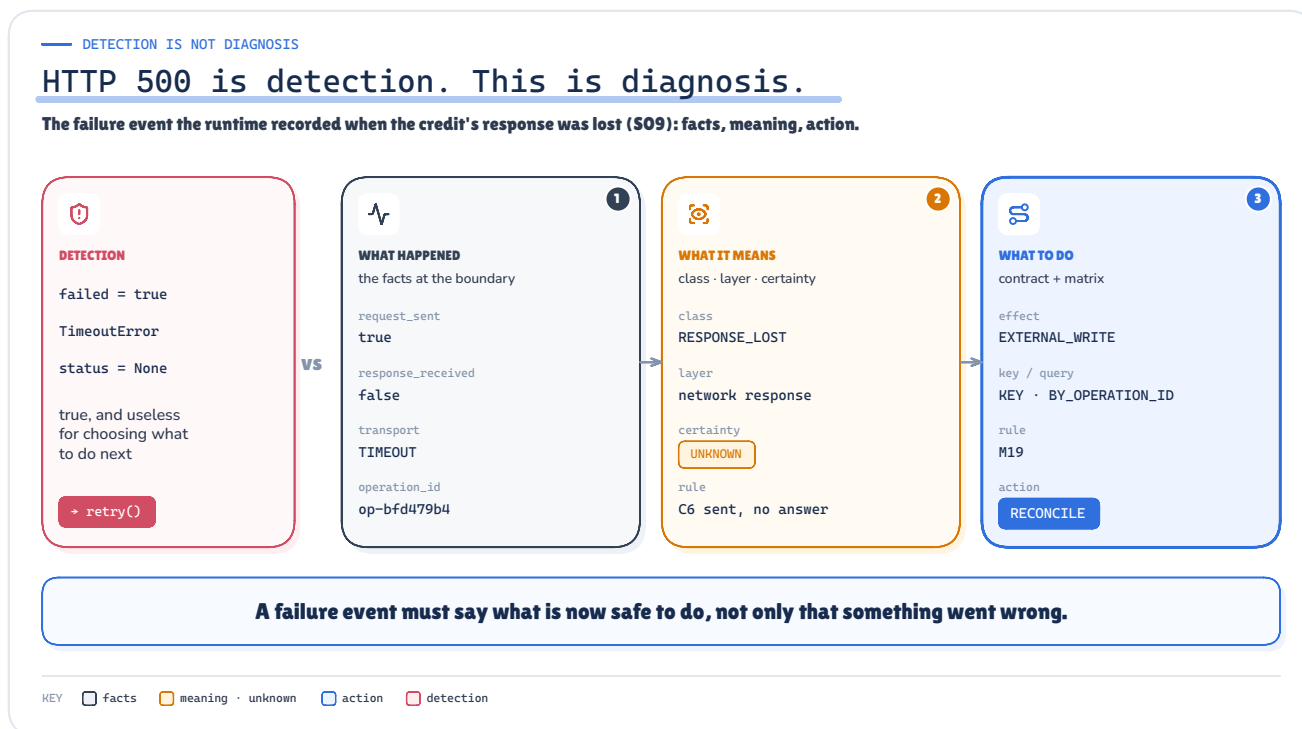
Observability creates evidence. Classification turns evidence into operational state. Reliability chooses the next safe action. Evals verify that both the behaviour and the recovery are still correct.

2 • Why “the agent failed” is insufficient

Consider the credit step of the support agent. A customer was charged twice; the agent decided to credit the duplicate charge, the policy engine allowed it, and the runtime sent `POST /credits`. Here are five ways that call can “fail”, each of which a typical runtime surfaces as an exception:

1. The connection was refused. Nothing was delivered. A retry cannot double anything.
2. The provider answered `422 CHARGE_DISPUTED`. It refused on a business rule, before any effect. A retry will get the same answer.
3. The provider committed the credit and the response was lost. The runtime saw a timeout. A retry without a key credits the customer twice.
4. The provider stalled, the request's deadline passed, and the provider refused to execute it late. The runtime saw the same timeout. A retry is now correct, and *not* retrying leaves the customer uncredited.
5. The worker process was killed after sending the request. Nothing in memory survived. Whether the credit happened is in the provider's ledger, not in the runtime.

Cases 3 and 4 are indistinguishable at the moment of the timeout. That is not a property of agents. It is the oldest property of distributed systems, and the reason gRPC documents that `DEADLINE_EXCEEDED` “may be returned even if the operation has completed successfully” ^[2], and why RFC 9110 says a client “SHOULD NOT automatically retry a request with a non-idempotent method unless it has some means to know that the request semantics are actually idempotent ... or some means to detect that the original request was never applied” ^[7]. What agents add is **more layers that fail** (retrieval, a probabilistic decision, structured output, tool selection, arguments, policy), **more side effects per run**, and a temptation to treat every failure as something the model can be asked to try again.



RECORDED Figure 1. Detection says that something failed. Diagnosis says what is now safe to do. The right-hand record is the failure event the runtime recorded in S09.

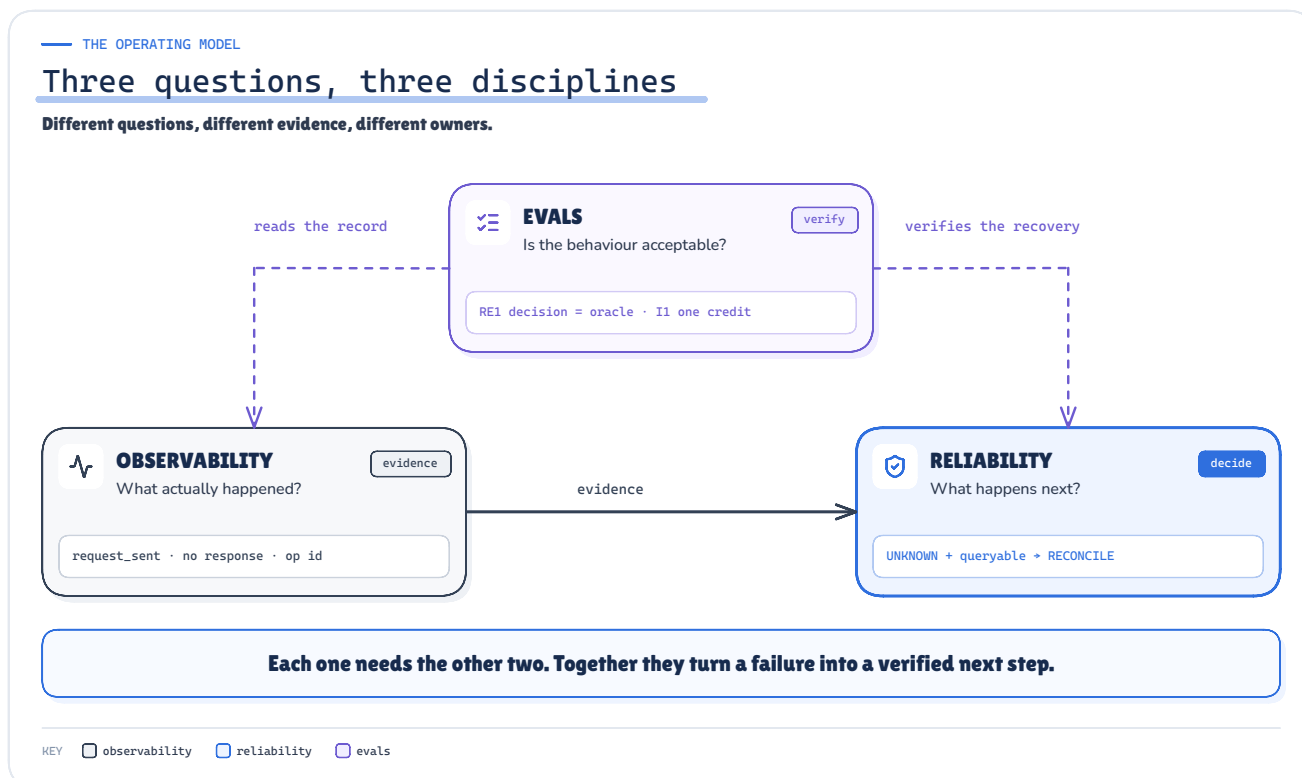
Recorded: S09's failure event, field by field · run 2026-10-07-recorded

3 · Three questions, three disciplines

The disciplines answer different questions, need different evidence, and usually have different owners:

	Question	Evidence	Typical owner	Failure if missing
Evals	Is the behaviour acceptable?	labelled cases, invariants, recorded runs	AI / product engineering	regressions ship; recovery bugs hide behind idempotency
Observability	What actually happened?	per-attempt facts, correlated across processes	platform / SRE	every incident is a reconstruction project
Reliability	What should the system do next?	facts + tool contract + persisted state	platform / runtime	retries duplicate effects, or give up on recoverable work

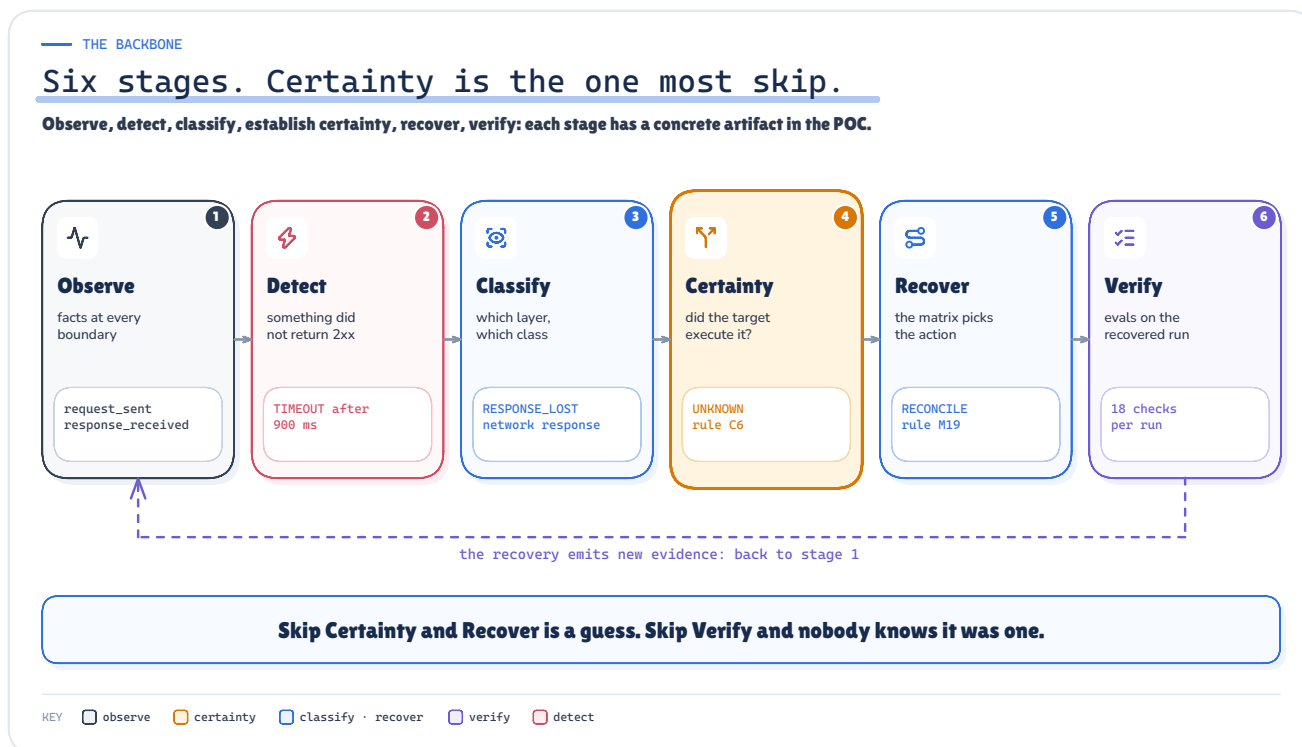
They depend on each other. A recovery policy without observability guesses. Observability without a policy produces dashboards and no action. Evals without both measure answer quality and miss the run that credited a customer twice while producing a perfect answer.



ARCHITECTURE Figure 2. Three questions, three disciplines. Evidence flows from observability to reliability; evals verify the recovery and read the record.

Architecture: our synthesis; the example strings are S09's recorded fields

The backbone of the rest of this edition is one loop:



ARCHITECTURE

RECORDED

Figure 3. The backbone. Each stage has a concrete artifact in the POC; the recovery itself is observed and evaluated again.

Architecture + recorded: each stage's artifact is S09's · run 2026-10-07-recorded

4 · Where this sits in the series

This article does not introduce another box in the platform. The capstone's layers stay as they are [P1]; what this article adds is a **cross-cutting operational plane**: every layer emits evidence, fails in its own way, and recovers differently.



ARCHITECTURE Figure 4. Where it lives. Each layer of the capstone platform with the failure classes it produces and its default recovery; one operational plane across them, and the evidence underneath.

Architecture: the capstone's layers; failure classes from recovery/taxonomy.py

How it relates to what the series already showed:

- **F2 · Layered Agent Platform** proved the mechanisms: checkpoints, resume after SIGKILL, a stable operation id, trace continuity, a model swap. Its E4 experiment counted 3 duplicate rollbacks in the monolith and 0 in the layered platform (**historical**). This article asks the next question: *given the evidence in hand, which mechanism should the runtime invoke?* F2 never had to choose: its backend honoured every key.
- **T2 · Authorization & Policy** decides whether an action is allowed. Here, `AUTHORIZATION_DENIED` is shown to be a different *operational* category from `TOOL_UNAVAILABLE`, with different recovery semantics: terminal, never retried.
- **T3 · Human-in-the-Loop** designed approval and intervention. Here, `REQUIRES_HUMAN` appears as a *recovery state*: what the runtime does when it cannot establish enough certainty to continue safely.

- **T5 • Observability & Governance** showed that execution lineage can prove what happened; its E12 pair counted 1 production change with an idempotency key and 2 without (**historical**). Here, telemetry exists to establish the operational facts a recovery decision needs, not to prove a story to an auditor.
- **P1 • the capstone** placed evaluations in the control plane and ran a lookup-versus-resend switch (R10; a fresh key per attempt gave 2 rollbacks, **historical**). Here the switch becomes a preregistered policy over 16 failure classes, and its correctness becomes something an eval can check.

The new evidence in this article is about the decision, not the mechanism: classification, execution certainty, the cases where an idempotency key does not help, reconciliation that can come back “absent”, “duplicate” or “unavailable”, and evals that catch a broken recovery layer.

5 • Failure detection versus failure diagnosis

Detection is noticing that something did not go as expected: an exception, a non-2xx status, a timeout, a missing heartbeat. **Diagnosis** is establishing what that means for the next action. Production reliability needs the second, and it needs it in a form a program can act on, not a stack trace a human reads at 3 a.m.

The POC records every failure as a `FailureEvent` (`recovery/classify.py`). The fields are the minimum that recovery needs:

```
{
  "run_id": "run-...", "step": "credit", "operation": "tool:issue_credit", "component": "tool",
  "failure_class": "RESPONSE_LOST", "layer": "network response",
  "attempt_id": "att-2130cc38", "operation_id": "op-bfd479b4", "idempotency_key": "op-bfd479b4",
  "request_sent": true, "response_received": false, "transport": "TIMEOUT", "http_status": null,
  "execution_certainty": "UNKNOWN", "certainty_rule": "C6",
  "certainty_basis": "sent, and no answer came back",
  "side_effect": "EXTERNAL_WRITE"
}
```

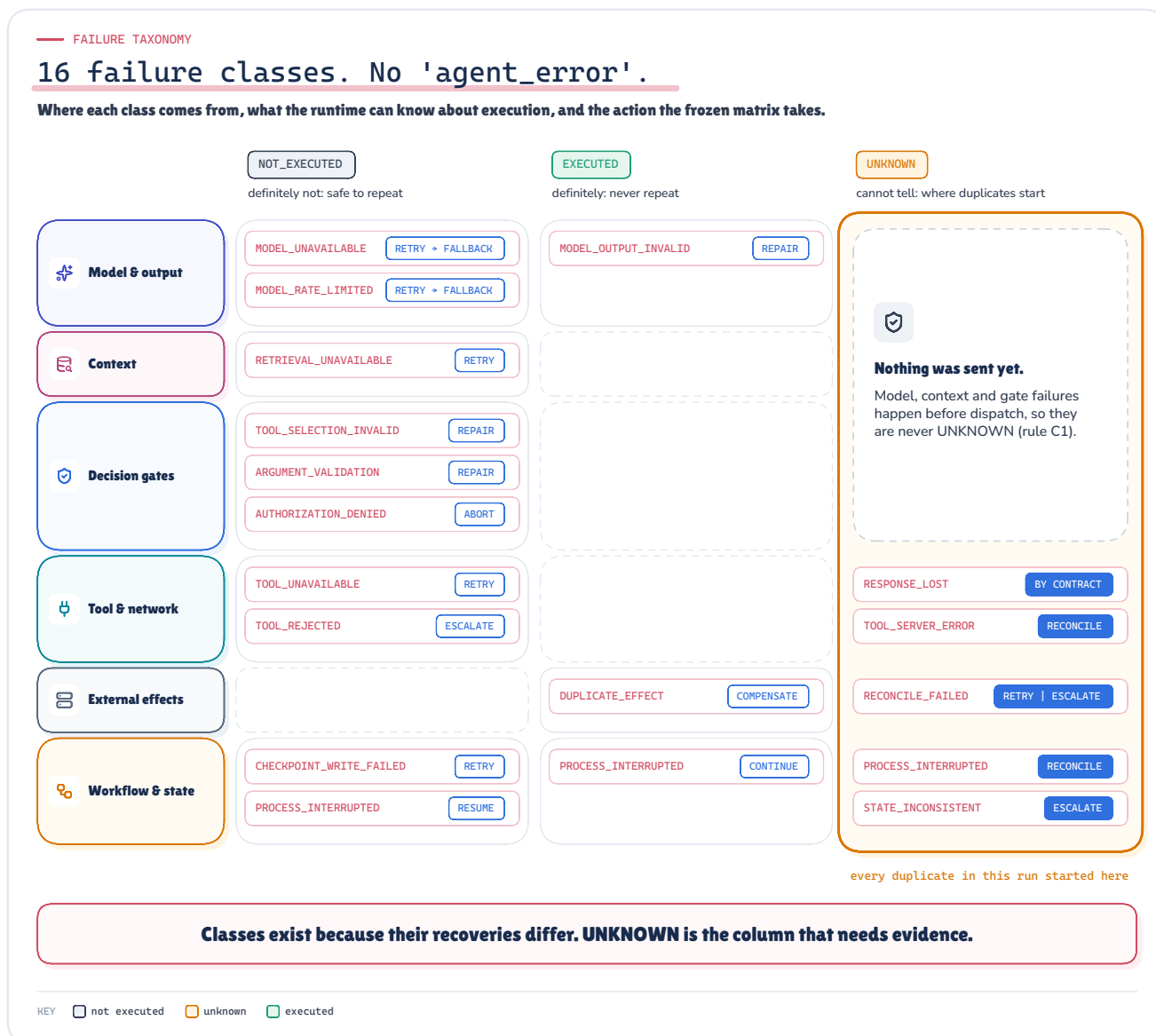
The decision that follows is a separate record, with the rule that produced it and the full situation it was computed from, so that it can be recomputed later from the matrix alone (invariant I12):

```
{"kind": "decision", "step": "credit", "failure_class": "RESPONSE_LOST", "certainty": "UNKNOWN",  
  "action": "RECONCILE", "rule": "M19",  
  "why": "outcome unknown on a write: ask the system of record before doing anything else",  
  "situation": {"class": "RESPONSE_LOST", "certainty": "UNKNOWN", "effect": "EXTERNAL_WRITE",  
    "idempotency": "KEY", "status_query": "BY_OPERATION_ID", "compensation": "NONE",  
    "key_fresh": true, "retries": 0, "repairs": 0, "reconciles": 0,  
    "fallback_available": true}}
```

Two design choices follow from the definition. First, **the client never decides:** `toolclient.py` only records evidence (was the request sent, how did transport end, was a response received, status, headers). Interpretation lives in one deterministic module. Second, **a failure event is about one operation attempt**, not about “the run”: a run can have a retried model call, a reconciled write and an escalated message, each with its own class and certainty.

6 • Failure taxonomy

The taxonomy has 16 failure classes across the layers of the platform. It is not a catalogue of everything that can go wrong; each class exists because its recovery differs from its neighbours’.



ARCHITECTURE Figure 5. The failure taxonomy as the POC implements it: each class with its layer, the certainty it usually carries, the frozen matrix’s default action and the scenario that exercises it. Implemented: the taxonomy and the frozen matrix's default actions; scenario ids from the oracle

The full matrix, with the evidence that identifies each class:

Failure layer	Class	Observable evidence	Execution certainty	Retry meaningful? safe?	Reconcile?	Compensation?	Human?	Verified by
retrieval	RETRIEVAL_UNAVAILABLE	503 + Not-Executed from the index	NOT_EXECUTED	yes · yes (a read)	no	n/a	after budget	RE1, OE1
model invocation	MODEL_UNAVAILABLE, MODEL_RATE_LIMITED	503 / 429 from the gateway	NOT_EXECUTED	once · yes; then fallback	no	n/a	after fallback	RE1
model output	MODEL_OUTPUT_INVALID	200, content is not the output contract	EXECUTED (the model ran)	no · regenerate with the error	no	n/a	after repair + fallback	RE1, TR1
tool selection	TOOL_SELECTION_INVALID	proposal names a tool outside the task's allow-list	NOT_EXECUTED (blocked)	no · repair once	no	n/a	after repair	I4, RE1
tool arguments	ARGUMENT_VALIDATION	schema or semantic check fails (wrong charge, wrong amount)	NOT_EXECUTED (blocked)	no · repair once	no	n/a	after repair	I4, RE1
authorization	AUTHORIZATION_DENIED	policy decision DENY with rule id	NOT_EXECUTED	no · retrying asks the same question	no	n/a	yes, via approval (T3)	I3, RE1
tool transport	TOOL_UNAVAILABLE	ECONNREFUSED; 503 shed; 408 deadline refusal	NOT_EXECUTED	yes · yes	no	n/a	after budget	RE1, RE4
tool execution	TOOL_REJECTED	documented pre-execution 4xx (dispute)	NOT_EXECUTED	no	no	n/a	yes	RE1
tool execution	TOOL_SERVER_ERROR	5xx after delivery	UNKNOWN	only after reconciliation	yes	maybe	if irreducible	I6, RE4
network response	RESPONSE_LOST	sent, timeout or reset, no response	UNKNOWN	depends on the tool contract	yes, if queryable	maybe	if irreducible	I6, RE1, RE4
downstream dependency	RECONCILE_FAILED	status query fails	UNKNOWN	keyed retry if the key is fresh	again, then stop	maybe	yes	RE1
external side effect	DUPLICATE_EFFECT	the provider holds more than one effect for our operation	EXECUTED	no	done	if the tool has an undo	if not	I1, I2, RE3
orchestrator	PROCESS_INTERRUPTED	a fresh worker finds the step in progress	from the journal (C7–C9)	depends on certainty	if UNKNOWN	—	—	I7, I8, RE1
workflow persistence	CHECKPOINT_WRITE_FAILED	the store refused the intent or result write	NOT_EXECUTED (intent) / EXECUTED (result)	intent: yes · result: fail-stop	result: on resume	—	after budget	RE1
workflow state	STATE_INCONSISTENT	the journal contradicts itself	—	no	no	no	yes	(not exercised)

The classes are the POC's, and the POC's tool set is small. A production taxonomy will be larger (quota errors, partial batch success, conflict on optimistic concurrency, schema drift in a tool's response), but the rule for adding a class is the same: add it when its recovery differs, never to make a dashboard prettier.

7 · Execution certainty

Execution certainty (our term): what the runtime knows about whether the target system executed one operation attempt. Three values: `NOT_EXECUTED` (definitely not), `EXECUTED` (definitely), `UNKNOWN` (cannot be established from the evidence in hand).

It is deliberately *not* a statement about success. A model call that returned unparseable output `EXECUTED`; the problem is its output, and the recovery (regenerate it) follows from that. A refund that the provider rejected for a dispute was `NOT_EXECUTED`; the recovery (escalate) follows from that and from the class. Certainty and side-effect semantics are kept apart because the action needs both: `UNKNOWN` on a read is harmless; `UNKNOWN` on a payment-like write is the whole problem.

The closest prior art is Jepsen, which records each operation as `:ok`, `:fail` (“didn’t take place”) or `:info` (“if we’re not sure ... might or might not have taken place”), and converts an exception into `:info` unless domain knowledge proves otherwise ^[1]. Jepsen uses this to check databases, not to recover; the mapping onto a runtime’s next action is ours.

Certainty is established by eleven ordered rules, frozen in `config/recovery-matrix.toml` with the policy:

Rule	Evidence	Certainty	Why
C1	<code>request_sent = false</code>	<code>NOT_EXECUTED</code>	blocked by validation or policy, or the pre-dispatch checkpoint failed
C2	transport <code>CONNECT_REFUSED</code>	<code>NOT_EXECUTED</code>	no byte was delivered
C3	response 2xx	<code>EXECUTED</code>	the provider says so
C4	response status in the tool’s documented pre-execution set	<code>NOT_EXECUTED</code>	the provider rejects before any effect
C5	503 with <code>Not-Executed: true</code>	<code>NOT_EXECUTED</code>	shed before acceptance
C6	sent, no response (timeout, reset)	UNKNOWN	the far side may or may not have run
C7	on resume: intent recorded, no result	UNKNOWN	a worker died in flight
C8	on resume: result recorded	<code>EXECUTED</code>	recorded before the interruption
C9	on resume: no intent recorded	<code>NOT_EXECUTED</code>	intent is always written before dispatch
C10	reconciliation, after the provider-enforced request deadline	<code>EXECUTED</code> if found, <code>NOT_EXECUTED</code> if absent	the system of record is authoritative once the provider will no longer accept the request
C11	any other response (5xx)	UNKNOWN	a server error after delivery says nothing about the effect

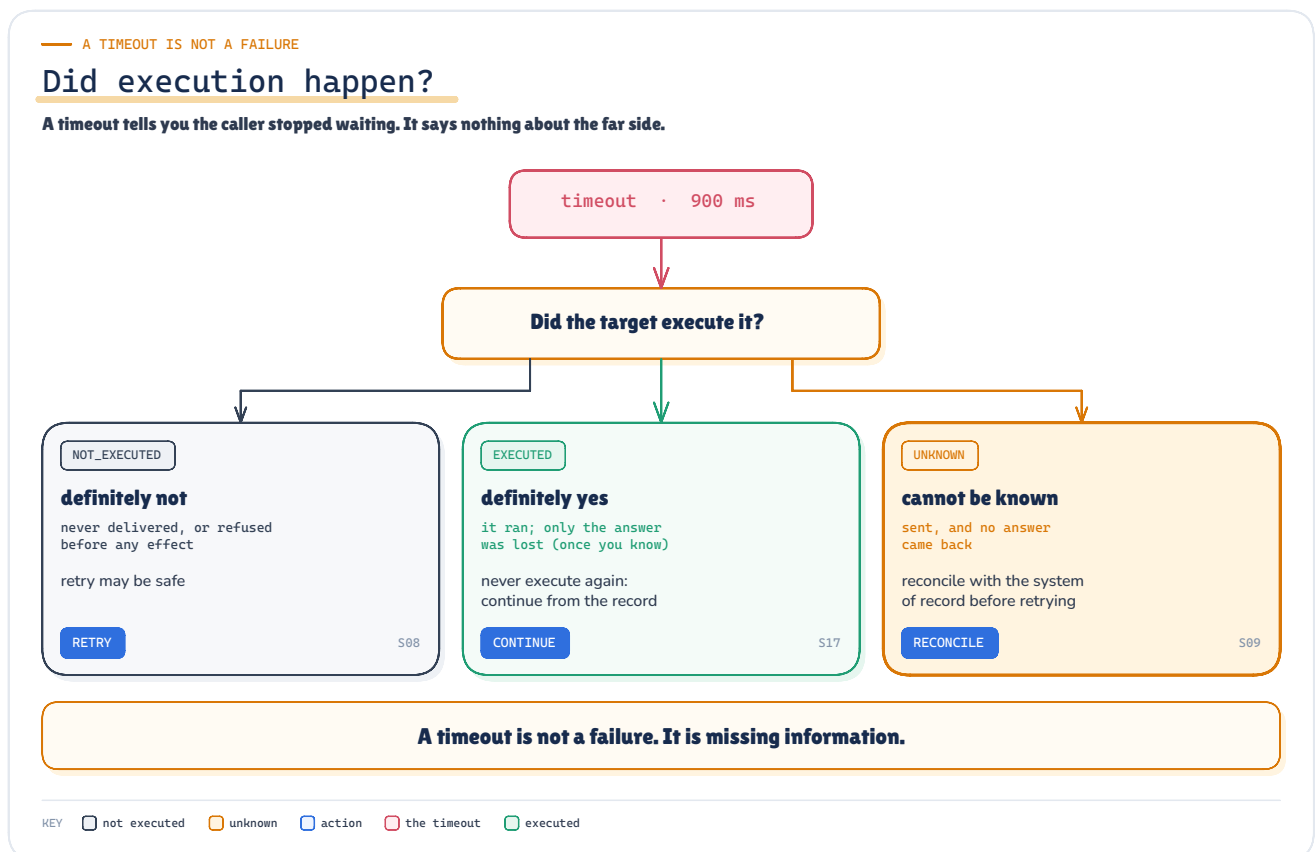
C4 is only as good as the provider’s documentation. Stripe, for example, documents that it saves an idempotent result only once execution has begun, and that a request failing validation is not saved ^[9]: that is the kind of statement a C4 entry needs. Where a provider does not document it, the status belongs under C11.

C10 has a precondition that matters: a reconciliation query that returns “not found” is only conclusive if the original request **can no longer arrive**. A request delayed in a queue or a retrying proxy can land after the query. The POC makes this concrete: every write carries a deadline (`x-Deadline-Epoch-Ms`), the providers refuse work whose deadline has passed, and the reconciler waits for the deadline before it asks. Without a deadline the provider enforces, “absent” is only “absent so far”: a timeout on the caller’s side says nothing about whether the far side is still working. Two further preconditions hold in the POC and need checking in production: the system of record keeps operation history for the whole recovery horizon, and its status query reflects its own writes. An eventually consistent one can answer “unknown” for a while.

8 · A timeout is not a failure

The distributed-systems pre-read of this series put it in one line: *a timeout is not a failure; it means the caller does not know the outcome*. Four different things hide behind “the call failed”:

	What happened	Certainty	Safe next step
request failure	never delivered (refused, DNS, closed port)	NOT_EXECUTED	retry
execution failure	delivered and refused before any effect	NOT_EXECUTED	depends on the class: fix, escalate or abort; retrying is pointless
response failure	executed, the answer was lost	EXECUTED (but the caller does not know it)	continue from the provider's record
effect uncertainty	sent, no answer, cannot tell which of the above	UNKNOWN	reconcile before anything else



ARCHITECTURE Figure 6. A timeout tells you the caller stopped waiting. The certainty decides between retry, continue and reconcile.

Architecture: certainty rules C1–C11 of the frozen matrix; scenario ids from the oracle

This is not new to agents, and mature stacks already behave this way. gRPC retries transparently “only if gRPC is certain the RPCs have not been processed by a server”, and stops for good once response headers arrive ^[3]. Marc Brooker’s rule is shorter: a timeout or failure “doesn’t necessarily mean that side effects haven’t happened” ^[11]. The Builders’ Library describes the case this article is built around: when it is not clear whether a provisioning request ran, “simply retrying the request could result in multiple workloads ... the provisioning process has to perform a reconciliation” ^[10].

9 • Unknown outcome is a first-class state

Most runtimes model an attempt as `SUCCESS | FAILED`. The POC’s run and step state is richer, and the extra states are what make recovery decidable:

```

NOT_STARTED → INTENT_RECORDED → DISPATCHED → { EXECUTED | NOT_EXECUTED | UNKNOWN }
UNKNOWN → RECONCILING → { EXECUTED → CONTINUE | NOT_EXECUTED → RETRY | DUPLICATE → COMPENSATE |
FAILED → ... }
terminal: COMPLETED · DENIED · REQUIRES_HUMAN · FAILED

```

`UNKNOWN` is never silently converted into `FAILED`. Converting it is exactly the X1 mutant (“a timeout is a failed call”), and the measured consequences are in §31. It is also never converted into `EXECUTED` because a retry eventually succeeded: in S20 the runtime could not reconcile, retried under the same key, and the provider replayed its stored answer; the effect was confirmed by the replay, not assumed.

What makes `UNKNOWN` dangerous is the side effect behind it. The POC’s four tools were chosen to cover the four contracts that matter:

Tool	Effect	Idempotency	Status query	Undo
<code>lookup_charges</code>	none (read)	—	—	—
<code>issue_credit</code> (payment-like)	external write	key , 24 h of provider time	by operation id	none
<code>create_ticket</code>	external write	none (the API ignores the key)	by our reference	void the ticket
<code>send_notification</code>	external write	none	none	none

One design note on the credit tool. Real card-refund APIs cap the refunded total at the charge amount, so a second full refund of the same charge is rejected by the provider’s own business rule. That cap is itself a domain invariant enforced by the provider, and a good one. Account credits, payouts, transfers, messages, tickets and deployments usually have no such cap, which is why the POC uses an account credit (`experiments/DEVIATIONS.md` D1 records the change, made before any run).

10 • The observability model: what recovery needs to see

The question for this section is narrow on purpose. Not *what is observability* (logs, metrics and traces are assumed), but: **what must be observed in an agent execution so that the runtime can make a correct recovery decision, and an operator can check it afterwards?**

The answer is a small set of facts per operation attempt, recorded at the boundary where they become true:

Group	Fields (POC names)	Where it becomes true	Used by
identity	<code>run_id</code> , <code>step</code> , <code>attempt_id</code> , <code>operation_id</code> , <code>trace_id</code> , <code>span_id</code> , <code>worker</code>	before the attempt	correlation (I8), idempotency (I10)
configuration	model and prompt-template version, policy version, matrix version, tool contract	at the step	regression analysis, release gates
context	retrieved document ids, versions and scores	after retrieval	retrieval evals, grounding
decision	proposed tool, a hash of the proposal, validation result, policy decision id and rule	at each gate	I3, I4, TR1
dispatch	<code>request_sent</code> , idempotency key, request deadline	when bytes leave	certainty C1, C2, C6
outcome	<code>response_received</code> , transport, status, <code>Not-Executed</code> , replayed, external id	when the answer arrives (or does not)	certainty C3–C5, C11
durable state	intent recorded, result recorded, checkpoint position	in the journal	certainty C7–C9, I7
diagnosis	failure class, layer, certainty, certainty rule	at classification	I5, RE1, RE4
decision	recovery action, matrix rule, the full situation	at the decision	I12, RE1
reconciliation	query, result (found / absent / duplicate / failed)	at reconciliation	certainty C10, RE4
cost	tokens, wall time, reconciliation queries	throughout	SLOs

What is **not** on that list matters as much. There is no model reasoning. Operationally useful decision information is the *observable* decision (which tool, which arguments, which documents it cited, which gate said no), not hidden chain-of-thought, which is neither reliable as an explanation nor safe to retain. There are no raw prompts, no customer e-mail addresses and no card numbers; §16 covers how the POC keeps them out.

11 • Correlation: one execution story

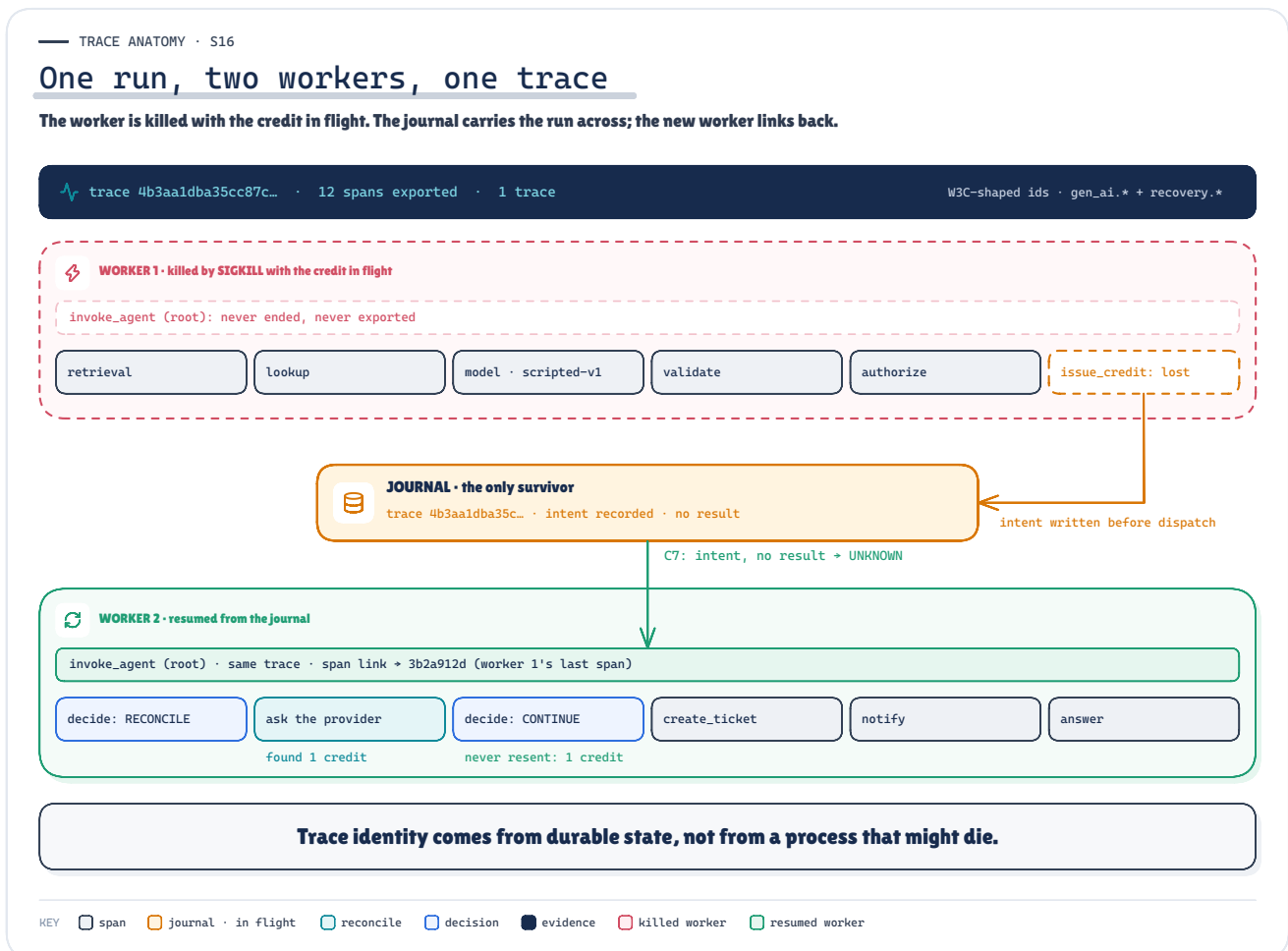
A single customer request became, in S16, this chain: retrieval, a model call, validation, authorization, a credit request, a SIGKILL, a new worker, a reconciliation query, a recovery decision, a ticket, a message and an answer. The run is only diagnosable if every one of those can be put back in order under one identity.

The POC uses W3C-shaped identifiers (a 16-byte trace id, 8-byte span ids) ^[32] and three rules:

1. **The run identity lives in durable state, not in a process.** The journal's `run` row holds the trace id; a resumed worker reads it back instead of starting a new trace.
2. **A resumed worker links to the interrupted one.** OpenTelemetry's span links exist for exactly this kind of causal relationship that is not parent-child ^[26]; the resumed root span carries a

link to the last span the killed worker exported (`recovery.link.reason = resumed after interruption`).

3. **Retries are attempts under one operation**, not new operations. Each attempt has its own attempt id and span, and all of them carry the same operation id. OpenTelemetry’s HTTP conventions model the same distinction: one span per physical attempt, with a resend count [25].



RECORDED Figure 7. S16 as the telemetry recorded it. Worker 1 was killed with the credit in flight: its root span never ended and was never exported, and the in-flight tool span was lost with it. Worker 2 restored the trace id from the journal and linked back.

Recorded: every exported span of S16 under the classified runtime · run 2026-10-07-recorded

Two properties of the recorded run are worth stating plainly, because they are properties of real telemetry, not of the POC:

- **A killed process loses its open spans.** S16’s first worker exported the spans that had ended and lost the rest, including the root and the in-flight tool call. Its children survive as orphans. The trace still reconstructs because the journal, not the tracer, is the system of record for “the credit was in flight”.

- **Traces are sampled.** OpenTelemetry is explicit that with head sampling “you cannot ensure that all traces with an error within them are sampled” [31]. Recovery must never depend on a span having been exported. The POC exports every span, but no decision reads a span.

The naive runtime starts a new trace when a worker restarts, a common default when the trace context lives only in memory. That split 4 of its runs into two traces (every scenario with a SIGKILL); the classified runtime split 0 (invariant I8).

12 · Logs, metrics, traces and structured events

Each signal has one job here, and none of them is the system of record for recovery:

Signal	Job in this architecture	Not for
structured events (journal)	the facts recovery reads: intent, result, failure events, decisions	dashboards; it is per-run, not aggregated
traces	the timeline across processes, and where time went	recovery decisions (sampled, lossy on crash)
metrics	rates and SLOs: unknown-outcome rate, duplicate suppressions, escalations	explaining a single run
logs	free-text detail for humans	anything a program must parse

A recurring trap deserves a sentence of its own. OpenTelemetry’s recording-errors guidance says that “errors that were retried or handled ... SHOULD NOT be recorded on spans or metrics that describe this operation” [28]. That is correct for the logical operation, and it means a run that reconciled its way out of a lost response can look perfectly healthy in a trace view. If recovery is not recorded as its own fact, it is invisible exactly when it worked. The POC records a `recovery <action>` span and a decision event for every decision, and the span status vocabulary itself (`Unset`, `Error`, `ok`) has no way to say “unknown”: certainty is an attribute, `recovery.execution_certainty`, not a status.

The attribute names follow the OpenTelemetry GenAI semantic conventions where a convention exists: `gen_ai.operation.name` (`invoke_agent`, `chat`, `execute_tool`), `gen_ai.request.model`, `gen_ai.tool.name`, `gen_ai.usage.*`, and `error.type` [22] [23] [29]. Those conventions are at **Development** stability, live in a repository without a tagged release at the time of writing, and will change. Everything recovery-specific has no convention and uses a `recovery.*` namespace. Nothing in the architecture depends on a vendor.

13 · Context and retrieval observability

Retrieval fails in two ways that look nothing alike in telemetry. An **unavailable index** is an infrastructure failure: a 503, a class (`RETRIEVAL_UNAVAILABLE`), a retry (S04). A **wrong result** is not a failure at all from the runtime’s point of view: the call returned 200 with three documents, and the problem only shows up later as an ungrounded answer.

That second case is why the retrieval step records the document ids, versions and scores it returned. An eval can say “the answer was not grounded”; only the record can say *why*: the relevant document was not in the top three. The real-model slice measured exactly this. The frozen BM25 retriever returned the labelled document in the top three for 13 of 16 cases; for MS-02, MS-05, MS-06 it did not, so no model could cite the right policy there, whatever it did. That is a retrieval defect, and it would have been misattributed to the model without the retrieval record. (It was found before the recorded run and deliberately not tuned; `experiments/DEVIATIONS.md` records it.)

14 · Tool and side-effect observability

For a tool call, the facts that matter are the ones that establish execution certainty, and each is recorded at the moment it becomes true:

intent recorded	journal, before dispatch: op id, key, deadline, request (redacted), trace id
request_sent	after the request bytes are handed to the socket
response_received	after a complete HTTP response, with status, Not-Executed, Idempotent-Replayed
result recorded	journal, after the outcome: certainty, external id, rule

Separating “sent” from “answered” is the single most useful thing a tool client can do for recovery, and it costs nothing. In the POC the provider client (`toolclient.py`) is deliberately dumb: it returns an `Evidence` record and never raises on a non-2xx. A client that raises one exception type for “connection refused”, “timed out after sending”, “422 dispute” and “500” has thrown away the information the runtime needs.

Side effects are observed in the **systems of record**, never in the agent’s own records. Every effect count in this article comes from the providers’ ledgers (credits per charge, open tickets per case, messages accepted per case). An agent that says “I created the ticket” has made a claim; the ticket system’s ledger is the evidence.

The Model Context Protocol, for comparison, separates protocol errors from tool execution errors (`isError: true`) but has no notion of an unknown outcome, and its `idempotentHint`, `readOnlyHint` and `destructiveHint` annotations are hints that clients “MUST consider ... untrusted unless they come from trusted servers” [4]. A tool contract good enough to recover with (effect, idempotency, status query, undo, pre-execution statuses) has to be owned by the platform, not inferred from a server’s self-description.

15 • Workflow and checkpoint observability

The journal is the workflow store, and it is also the most important observability surface for recovery, because it is the only one that survives a crash by design. §22 covers what it holds; for observability, two properties matter:

- **Everything is written before the step it describes.** An intent is durable before the request is sent; a result is durable before the worker moves on. A worker that dies mid-call therefore always leaves “this operation may be in flight” behind.
- **A failed write is a fact too.** In S22 the store refused the intent write; the runtime recorded `CHECKPOINT_WRITE_FAILED` with certainty `NOT_EXECUTED` (nothing was sent) and retried the write. In S23 the store refused the *result* write after the credit committed; the runtime recorded it as `EXECUTED`, stopped that worker on purpose (the run's only fail-stop: 1 across 30 classified workers), and the next worker reconciled.

16 • Telemetry governance: what must never be recorded

Observability that leaks is a liability. The POC plants three canaries in the simulated world: a test card number, the customer’s e-mail address in the message text, and a provider API token in configuration. Invariant I11 scans every run’s events, journal and spans for them. Every run of every runtime passed: the classified runtime’s I11 count was 25 of 25, and the same holds for the baselines because redaction is part of the shared telemetry, not of a recovery layer.

Three mechanisms keep them out:

1. **Minimise at the source.** The notification request carries a case id and a template, never an address: the provider resolves the recipient. The idempotency key is an opaque operation id, never a customer identifier, as Stripe also advises [9].
2. **Redact values, not identifiers.** The tracer redacts card numbers, e-mail addresses and token shapes in attribute and event values before anything is written, and never rewrites ids.
3. **Record decisions, not content.** The journal stores a hash of the proposal, the tool and its arguments (which here contain no personal data), the policy decision id and the rule. Prompt and completion capture is off, as the OpenTelemetry GenAI conventions recommend by

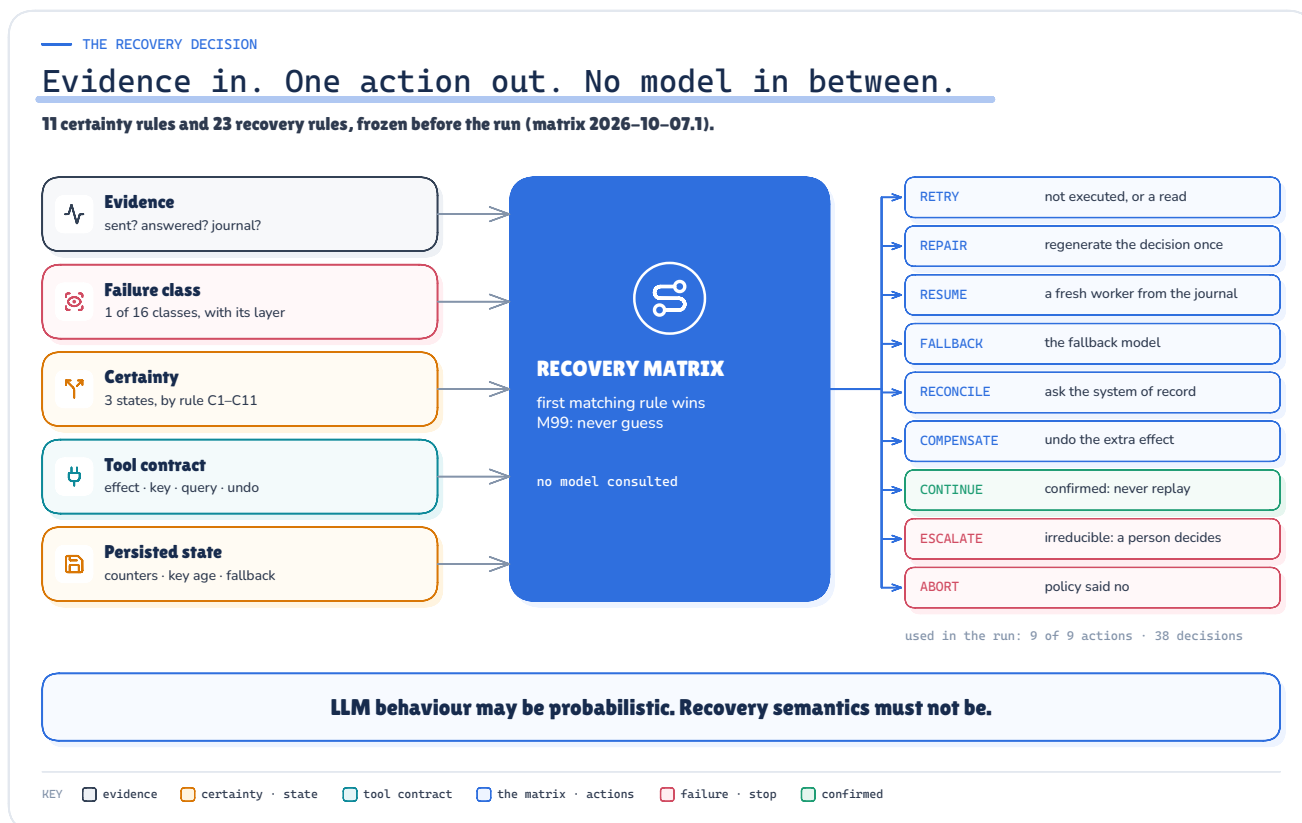
default ^[22] ^[34], and the OWASP logging guidance lists the categories that should never be logged at all ^[35].

T5 treated retention and evidence integrity at length; this article only needs the narrower rule: **the facts recovery depends on (ids, statuses, certainties, rules) contain no secrets, so there is never a reason to choose between recoverability and privacy.**

17 · Reliability is the policy from evidence to action

Reliability here is not “add retries”. It is a policy: a function from what the runtime knows to what it does next. The POC implements it as data, `config/recovery-matrix.toml`, frozen with the preregistration before any run and loaded by the runtime at start. 23 rules, first match wins, the last one a catch-all that escalates.

```
failure class + execution certainty + side-effect semantics + persisted state → action
(14 classes)      (C1-C11)                (tools.toml contract)      (counters, key age)      (9
actions)
```



ARCHITECTURE **MEASURED** Figure 8. The recovery decision. Five inputs, one deterministic matrix, nine actions. Every decision the runtime records names its rule, so it can be recomputed from the matrix and the recorded evidence alone.

Implemented: the frozen recovery matrix; usage counts from run 2026-10-07-recorded

The nine actions and what they mean in this runtime:

Action	Meaning	Typical trigger (rule)
RETRY	re-attempt the same operation, same operation id, same key	NOT_EXECUTED (M17), a read with UNKNOWN (M18), reconciled absent (M16), a fresh key (M20)
REPAIR	regenerate the model's decision once, with the validation error	MODEL_OUTPUT_INVALID, TOOL_SELECTION_INVALID, ARGUMENT_VALIDATION (M03)
RESUME	a fresh worker continues from the journal	NOT_EXECUTED on resume (M12); a result that could not be persisted (M11)
RECONCILE	ask the system of record about this operation	UNKNOWN on a write with a status query (M19)
CONTINUE	adopt the recorded or reconciled result; never replay	EXECUTED (M13)
FALLBACK	switch to the fallback model	the provider keeps failing (M07), output keeps failing (M04)
COMPENSATE	undo the extra effect, keep one	a duplicate on a tool with an undo (M14)
ESCALATE	stop and hand it to a person, with the evidence	a business rejection (M09), irreducible UNKNOWN (M21), no rule (M99)
ABORT	stop; the answer is no	AUTHORIZATION_DENIED (M02)

The classified runtime used all 9 actions across the run's 38 decisions, and every decision recomputed from the matrix (I12 held in 25 of 25 runs).

Deterministic, not delegated

No rule consults a model. That is a design decision, not an omission. The question “a payment request timed out after it was sent; should I send it again?” has an answer that depends on facts (was it sent, can the provider be asked, does the provider deduplicate, how old is the key) and none on judgement or language. A model asked that question will sometimes say yes. LLM behaviour may be probabilistic; recovery semantics must not be. Where judgement is needed, at an irreducible `UNKNOWN` or a business rejection, the matrix escalates to a person rather than to a model.

Recovery that is deterministic is also **testable**: the oracle for every scenario was written down before the run, and recovery evals compare the recorded decisions with it (§30).

18 · Retry semantics

A retry is correct when a re-attempt cannot change the outcome for the worse. That is a property of the evidence, not of the exception:

- **Safe**: definitely not executed (refused connection, documented pre-execution rejection, shed with `Not-Executed`), or the operation is a read.

- **Safe with a condition:** the provider deduplicates by key *and* the key is still inside its window; or reconciliation established, after the deadline, that the operation is absent.
- **Pointless if resent unchanged:** authorization denied, a business rule rejected it, the arguments failed validation, the tool selection was invalid. Invalid arguments and tool selections get one `REPAIR`, which regenerates the proposal; a policy refusal gets none. These are retried by the naive runtime and the idempotent runtime alike: 8 and 8 re-attempts of refusals no retry can change, against 0.
- **Unsafe:** UNKNOWN on a write with no reconciliation and no fresh key.

Retries of safe operations still need budgets and backoff with jitter to avoid amplifying an outage [11] [13] [14]; the POC keeps a budget of two per step and class and does not model load. The point here is the one RFC 9110 already makes for HTTP: a client “SHOULD NOT automatically retry” a non-idempotent request without “some means to know” it is idempotent “or some means to detect that the original request was never applied” [7]. Durable-execution engines encode the same rule as non-retryable error types [5] [6], and gRPC as a status-code contract: `UNAVAILABLE` retries the call, `FAILED_PRECONDITION` waits for the state to be fixed [2].

19 • Idempotency: necessary, not sufficient

An idempotency key turns “send it again” into “ask the provider for the answer to the request it already executed”. Stripe saves the status and body of the first request for a key and returns them for later ones, “including 500 errors” [9]; AWS builds a caller-provided client token into its API contracts and rejects a reused token with different parameters [10] [12]. The IETF draft for a standard `Idempotency-Key` header describes the timeout case directly and specifies conflict responses, but it is still a working-group draft (expired April 2026) [8].

In the POC the classified and idempotent runtimes derive the key from the operation, not the attempt: `op-<hash(run, step)>`, sent on every attempt (I10). The X7 mutant derives it from the attempt instead, a bug that is easy to write; it duplicated the credit in S20 and failed I10 in 24 of 25 scenarios.

What idempotency does **not** do, measured:



MEASURED Figure 9. The idempotent–retry runtime against the scenarios where a key is not enough. c, t, m: credits on the charge, open tickets, messages sent; the correct count for each is 1.

Measured: six scenarios × three runtimes, from the providers' ledgers · run 2026-10-07-recorded

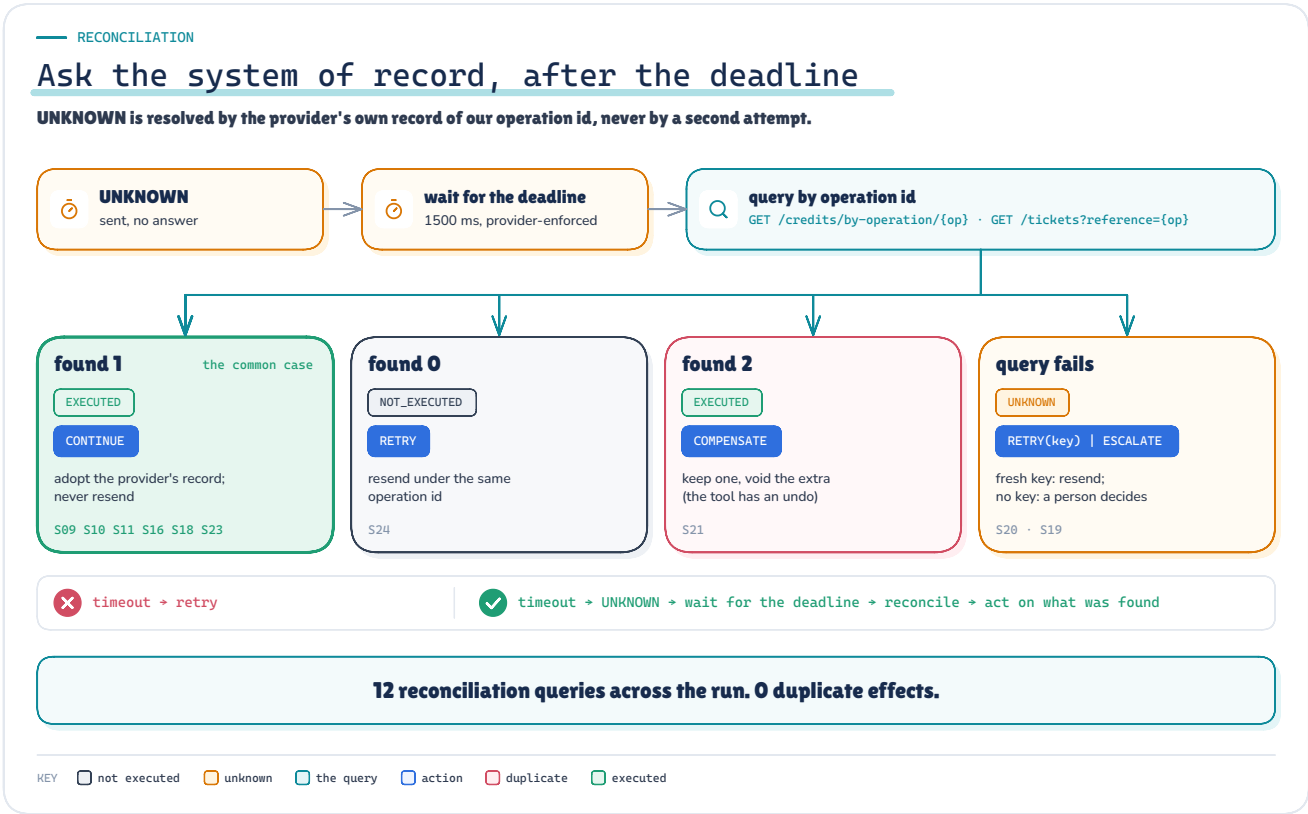
- A key protects one provider.** The ticket API ignores `Idempotency-Key`, and the messaging provider has no key at all. With the same retry logic, A1 duplicated tickets and messages in S11, S12, S19 and S21.
- A key protects a window.** Stripe documents that keys may be pruned after 24 hours and that a reused key after pruning is a new request^[9]. In S18 the worker died with the credit in flight and resumed after an outage longer than the window; A1 resent under the same key and the provider credited again. The classified runtime reconciled by operation id, which does not depend on the key's lifetime, only on the provider retaining the operation's history for the recovery horizon.
- A key does not tell you what happened.** In S10 every response was lost. A1's three attempts produced 1 credit (the key held) and a final answer that said the credit had **failed**. Duplicate suppression without certainty turns a duplicate into a false failure.

The idempotent runtime is the strongest generic baseline in this experiment, the mechanism F2 and T5 already measured. It duplicated an external effect in 5 of 25 scenarios (S11, S12, S18, S19, S21); the naive runtime in 10; the classified runtime in 0.

On terminology: none of this is *exactly-once execution*. Kafka’s documentation warns that exactly-once claims need reading in the fine print and that exactly-once delivery to external systems “generally requires cooperation with such systems” [15]; Temporal notes that an activity that completed but whose worker crashed before reporting will run again [16]. What the POC demonstrates is narrower and precise: **at-least-once attempts, a durable operation identity, duplicate suppression where the provider supports it, and reconciliation where it does not, giving effectively-once business effects in every one of the 25 measured scenarios (I1, I2)**. An unmeasured fault could still produce a duplicate, and S12 shows the case where the runtime cannot rule one out and says so.

20 • Reconciliation

When the certainty is UNKNOWN and the operation changes external state, the runtime asks the system of record before it does anything else (M19). The AWS Builders’ Library describes exactly this step for provisioning: retrying blindly “could result in multiple workloads”, so “the provisioning process has to perform a reconciliation” [10]; Kubernetes is built on the same idea at a larger scale, controllers that converge observed state to desired state [21].



MEASURED Figure 10. Reconciliation in the POC: wait for the deadline the provider enforces, query the provider by operation id or business reference, and let the answer pick the action. Implemented + measured: reconciliation outcomes and the scenarios that produced them · run 2026-10-07-recorded

The four outcomes all occurred in the recorded run:

Reconciliation result	Certainty (rule)	Action	Scenario
one effect for our operation	EXECUTED (C10)	CONTINUE, adopt its id (M13)	S09, S10, S11, S16, S18, S23
none, after the deadline	NOT_EXECUTED (C10)	RETRY under the same id (M16)	S24: the provider stalled and refused late; the retry committed once
more than one	EXECUTED (C10)	COMPENSATE if the tool has an undo (M14), else ESCALATE (M15)	S21: two tickets, one voided
the query itself failed	UNKNOWN	RECONCILE again, then a keyed retry if the key is fresh (M20), else ESCALATE (M21)	S20 (keyed retry, replayed), S19 (escalated)

Three implementation details carry most of the correctness:

- **Query by operation, not by content.** “Is there a credit of 42.50 on charge `ch_1042_b`?” cannot tell our credit from a legitimate earlier one. The operation id (or a business reference the runtime chose) can.
- **Wait for the provider’s deadline.** A “not found” only becomes `NOT_EXECUTED` once the original request can no longer land (C10). Without a deadline the provider enforces, an absent record during an in-flight request is a race, and a retry can still duplicate.
- **Reconciliation can fail.** It is a call to a downstream dependency like any other; its failure is a class (`RECONCILE_FAILED`) with its own rules, not a reason to fall back to retrying.

The cost is real and measured: 12 reconciliation queries across the 25 scenarios, each after waiting out a deadline. Recovery latency on an `UNKNOWN` write is therefore at least the request deadline (here 1,500 ms). That is the price of not guessing.

21 • Compensation

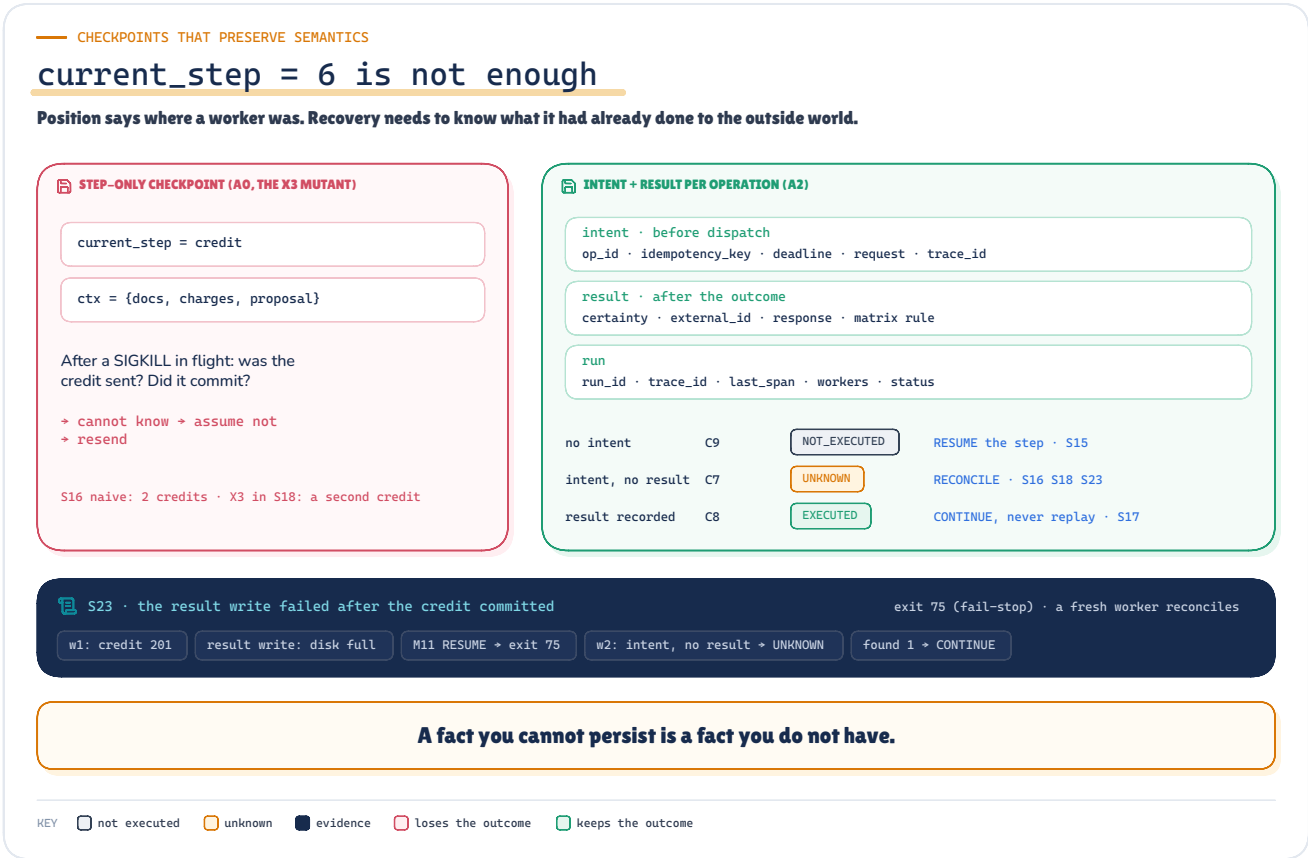
Compensation undoes an effect semantically; it does not restore the previous state. The original Sagas paper and the Azure pattern descriptions are explicit that a compensating transaction “does not necessarily return the database to the state” it was in, that it can itself fail, and that some steps cannot be compensated at all ^{[18] [20] [19]}.

The POC uses compensation in one narrow, measurable place: when reconciliation finds *more* effects than intended for one operation. In S21 the ticket provider committed the ticket twice internally (an upstream at-least-once retry) and lost the response; the runtime reconciled, found two open tickets for its reference, voided one and continued (M14). Had the duplicate been a credit,

the matrix escalates instead (M15): a second payment has no automatic undo, and reversing one is a business decision. Compensation is a property of the tool contract, so it lives in `tools.toml` (`compensation = "void_ticket"`), not in the agent.

22 · Checkpoints and resume: persist semantics, not position

F2 showed that a durable workflow can resume after a SIGKILL. The question here is narrower: **what must the checkpoint contain so that resume preserves semantics?**



RECORDED Figure 11. A step position tells a resumed worker where it was. Recovery needs to know what it had already done to the outside world.

Implemented + recorded: the journal schema (`recovery/journal.py`); S16 and S23 recorded · run 2026-10-07-recorded

The classified runtime persists, per operation:

Record	Written	Contents	Why
intent	before dispatch	operation id, idempotency key, request deadline, redacted request, trace id	a crash after this point is UNKNOWN , not NOT_EXECUTED
result	after the outcome	certainty, external resource id, provider response, the rule that decided	a crash after this point is EXECUTED : continue, never replay
step checkpoint	after each step	position and step outputs (documents, charges, proposal, policy decision)	resume without re-asking the model
run	at start and on change	run id, trace id, last exported span, worker count, status	trace continuity across workers

The resume rules follow (C7–C9): no intent means the step never dispatched (RESUME it, S15); an intent without a result means it may be in flight (RECONCILE, S16, S18, S23); a recorded result means it is done (CONTINUE without replay, S17). The naive runtime’s checkpoint has the position and the step outputs, which is what most frameworks persist; after a SIGKILL in flight it resent the credit (2 credits in S16). The X3 mutant, the classified runtime with a step-only checkpoint, re-dispatched in S16, S17, S18 and S23 and duplicated the credit in S18, where the key had expired.

One failure mode deserves a rule of its own: the result write fails *after* the effect committed (S23). The runtime knows the credit executed, but it cannot persist that, and a fact it cannot persist is a fact the next worker will not have. Continuing would leave the journal claiming the credit is in flight forever. The matrix stops the worker (M11, exit code 75) and lets a fresh one resume from the journal, which finds an intent without a result, reconciles, and continues. The naive runtime caught the same storage error as “the step failed”, retried the step, and credited twice.

23 • Human escalation as a recovery state

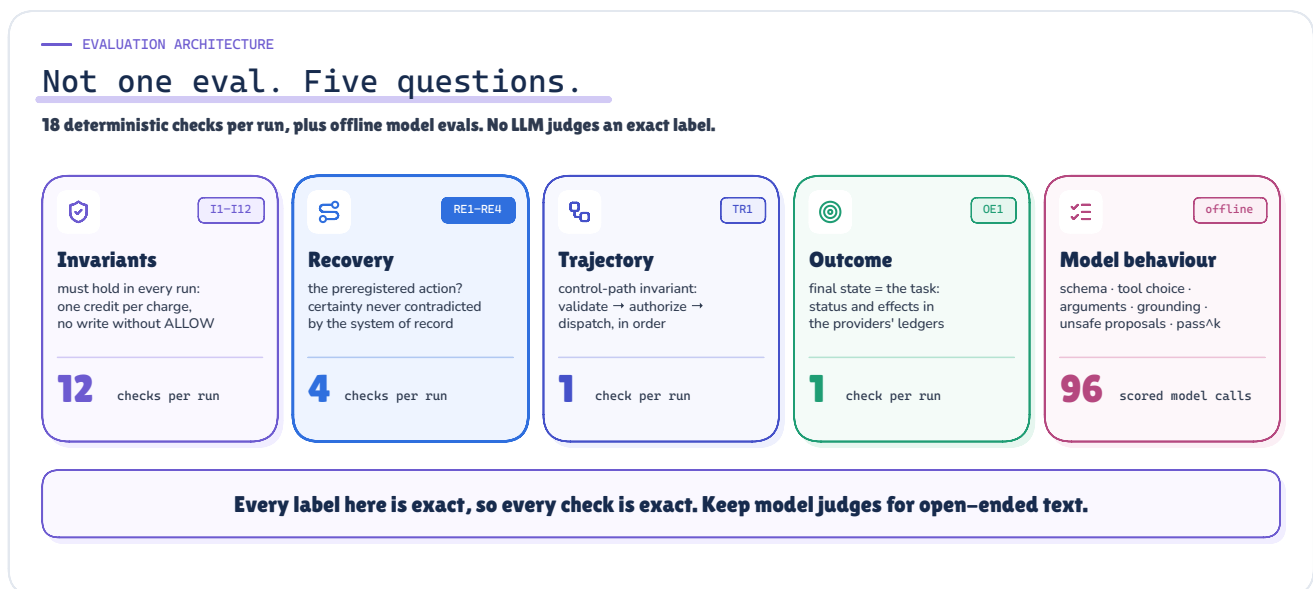
Escalation is not a failure of the recovery layer; it is one of its outputs. The classified runtime escalated in 3 scenarios, exactly the 3 the oracle required, and in none it did not (0 unnecessary escalations):

- **S12**, a message whose response was lost: no key, no status query. Nothing the runtime can do will establish whether the customer received it. Resending risks a duplicate message; not resending risks none. The run ends REQUIRES_HUMAN with the message marked **unknown**, not sent and not failed.
- **S13**, a credit rejected because the charge is under dispute: a business rule, not a fault.
- **S19**, a ticket whose response was lost while ticket search was down, twice: irreducible within the budget.

An escalation should carry what a person needs to decide quickly: the failure event, the certainty and its rule, what was reconciled and how, and what the runtime did *not* do. T3 designed the approval mechanics; here, `REQUIRES_HUMAN` is simply the terminal status of a run whose next safe action is not automatic. Authorization denial (S07) is different: it ends `DENIED` with an `ABORT`, because the question was answered; a person may still approve a higher credit, through the approval path, not through recovery.

24 • The evaluation architecture

“Evals” in agent work often means one thing: a judge model scoring answers. That is one tool, for one kind of question. The POC uses five families, each answering a different question about a run, and all of them except the model slice’s statistics are deterministic comparisons over recorded files.



ARCHITECTURE Figure 12. The five eval families of this article. Four run on every scenario run; the fifth gates model changes offline.

Implemented: the eval suite (recovery/evals.py) and the model-slice evaluators

The split mirrors how Anthropic’s guidance on agent evals separates the **transcript** (“also called a trace or trajectory”) from the **outcome** (“the final state in the environment”), and its advice to choose “deterministic graders where possible, LLM graders where necessary”^[36]. τ -bench grades outcomes the same way, by comparing the database state at the end of a conversation with the annotated goal state^[40]. What this article adds is the family in the middle: **recovery evals**, which grade the decisions a runtime made under failure.

The checks, as implemented in `recovery/evals.py`:

Check	Family	Question	Reads
I1	invariant	at most one committed credit per charge	credit ledger
I2	invariant	at most one open ticket and one message per case	ticket and message ledgers
I3	invariant	every credit dispatch has an ALLOW for <i>that</i> proposal; nothing after a DENY	events
I4	invariant	a proposal that failed validation is never dispatched	events
I5	invariant	every failure event has a specific class and a certainty	events
I6	invariant	an UNKNOWN write is not re-dispatched before reconciliation (except M16, M20)	events
I7	invariant	a write whose outcome was recorded is never dispatched again	events
I8	invariant	one trace per run; every resumed worker links back	spans
I9	invariant	the answer's claims equal the ledgers and rest on confirmed state	answer, ledgers, journal
I10	invariant	every attempt of a write carries the operation id as its key	events
I11	invariant	no canary in telemetry or the journal	events, journal, spans
I12	invariant	every decision recomputes from the matrix	events, matrix
RE1	recovery	the recorded decisions equal the preregistered oracle	events, oracle
RE2	recovery	the terminal status equals the oracle	answer, oracle
RE3	recovery	the effects equal the oracle	ledgers, oracle
RE4	recovery	no stated certainty is contradicted by the provider's records	events, access log, ledgers
TR1	trajectory	the path to the credit went retrieval → model → validation → authorization → dispatch	events
OE1	outcome	status and effects are those of a correct run	RE2 and RE3

25 • Deterministic invariants

Invariants are the cheapest evals and the most valuable. They need no labels, run on every execution, and turn a business rule (“a customer is never credited twice for one duplicate charge”) into a check that fails a build. The user-facing examples in this series’ brief map directly onto them:

Rule as a sentence	Check
A payment-like side effect must never execute twice.	I1
An authorization-denied tool must never execute.	I3
A resumed workflow must preserve run identity.	I8
A response claiming success must correspond to confirmed side-effect state.	I9
A tool call must satisfy its schema.	I4 (and the validation gate)
A checkpoint resume must not repeat an already-committed operation.	I7

The classified runtime failed 0 invariant checks across the 25 scenarios. The idempotent runtime failed 26 and the naive runtime 56; most of the baselines' failures are I5 (their failures carry no class or certainty, by design) and I10 for the naive runtime (no operation identity), and the rest are duplicates (I1, I2), a false claim (I9) and split traces (I8).

One invariant needed a correction after the run. I7's first implementation compared dispatches with the journal's result row, which a re-dispatch overwrites; it could not see the X3 mutant re-dispatching an already-recorded credit in S17. It now reads the append-only event sequence, every `eval.json` was recomputed from the recorded files, and nothing about A0, A1 or A2 changed (`experiments/DEVIATIONS.md` D3). It is a small example of why the evaluator needs its own negative controls: a check that has never failed has not been shown to work.

26 · Structured output, tool selection and arguments

These evals sit between the model and the runtime. In the scenarios they are gates (the runtime refuses to act on a proposal that fails them); in the real-model slice they are measurements:

- **Structured output.** Is the response JSON with the contract's keys and types? The models ran in JSON mode, not schema-constrained decoding, so schema violations stay observable. (With constrained decoding, structured-output validity becomes a property of the decoder; the semantic checks below still apply.)
- **Tool selection.** Is the chosen action the labelled one, from the task's allow-list?
- **Arguments.** For a credit: exactly the duplicate charge and its amount. Semantic checks matter more than schema here: `{"charge_id": "ch_1042_a", "amount": 42.50}` is schema-valid and wrong, because it credits the original charge, not the duplicate.
- **Unsafe proposals.** A credit where none is due, or on the wrong charge. Counted separately from “wrong”, because an escalation that set `requires_approval` on a dispute is wrong by the contract and harmless, while a credit on a pending authorization is neither.

The Berkeley Function Calling Leaderboard evaluates tool calls the same way, by parsing the call and checking it, and by executing it ^[41]. Exact labels make the comparison cheap and indisputable.

27 • Retrieval evals

Retrieval was evaluated as `recall@3` against a labelled relevant document per case, and grounding as “the answer cites the labelled document, and cites nothing it was not given”. RAGAS formalises related reference-free measures (faithfulness, answer and context relevance) for when labels do not exist ^[42]; with labels, exact recall is simpler and more honest.

The frozen retriever found the labelled document in the top three for 13 of 16 cases. For the three it missed, no model could produce a grounded citation, and both models’ grounding rates show it. This is where evals and observability meet: the eval says “grounding failed”; the retrieval record (document ids, versions, scores) says “because retrieval never surfaced the policy”, and the fix is in the index, not the prompt.

28 • Trajectory evals

A trajectory eval checks the observable *path*, not the hidden reasoning. Here it is a **control-path invariant**: an order that safety requires (authorize before dispatch), not a grade on how the agent chose its steps, which would be brittle for an agent that plans its own tool sequence. TR1 requires that the credit that was dispatched was preceded, in order, by retrieval, a model proposal with the same hash, a passed validation of that proposal, and an ALLOW for that proposal. The unsafe path it guards against is a short one:

```
safe:    retrieval → model(proposal p) → validate(p) → authorize(p) = ALLOW → dispatch(p)
unsafe:  retrieval → model(proposal p) → validate(p) → dispatch(p)           # the X5 mutant, after
a repair
```

The X5 mutant skips authorization for a repaired proposal, a plausible bug in any “retry the step” code path. It failed TR1 and I3 in the three scenarios that repair (S03, S05, S06), and passed everything else. Trajectory checks over recorded events need no access to model internals; LangChain’s `agentevals`, for example, matches recorded trajectories against reference ones in strict, unordered or subset modes ^[44].

29 · Outcome evals

OE1 asks the only question a customer cares about: is the world in the right state? Status and effect counts are read from the providers’ ledgers and compared with the oracle. Outcome evals are necessary and not sufficient, and §31 shows the case that proves it: a recovery bug that an outcome check cannot see.

30 · Recovery evals

Recovery evals check the decisions a runtime makes under failure (our term; the closest published practice is outcome-state grading in agent benchmarks and fault-injection testing in distributed systems). The questions they answer:

Question	Check
Did UNKNOWN choose RECONCILE rather than RETRY?	RE1 (the decision sequence), I6
Did a committed operation have one business effect?	I1, I2, RE3
Did resume preserve the original operation identity?	I10, I7
Was authorization denial treated as terminal rather than transient?	RE1 (ABORT), I3
Did reconciliation recover the correct external state?	RE4 (no certainty contradicted by the provider), RE3
Did the recovered workflow still satisfy the original task?	OE1, I9

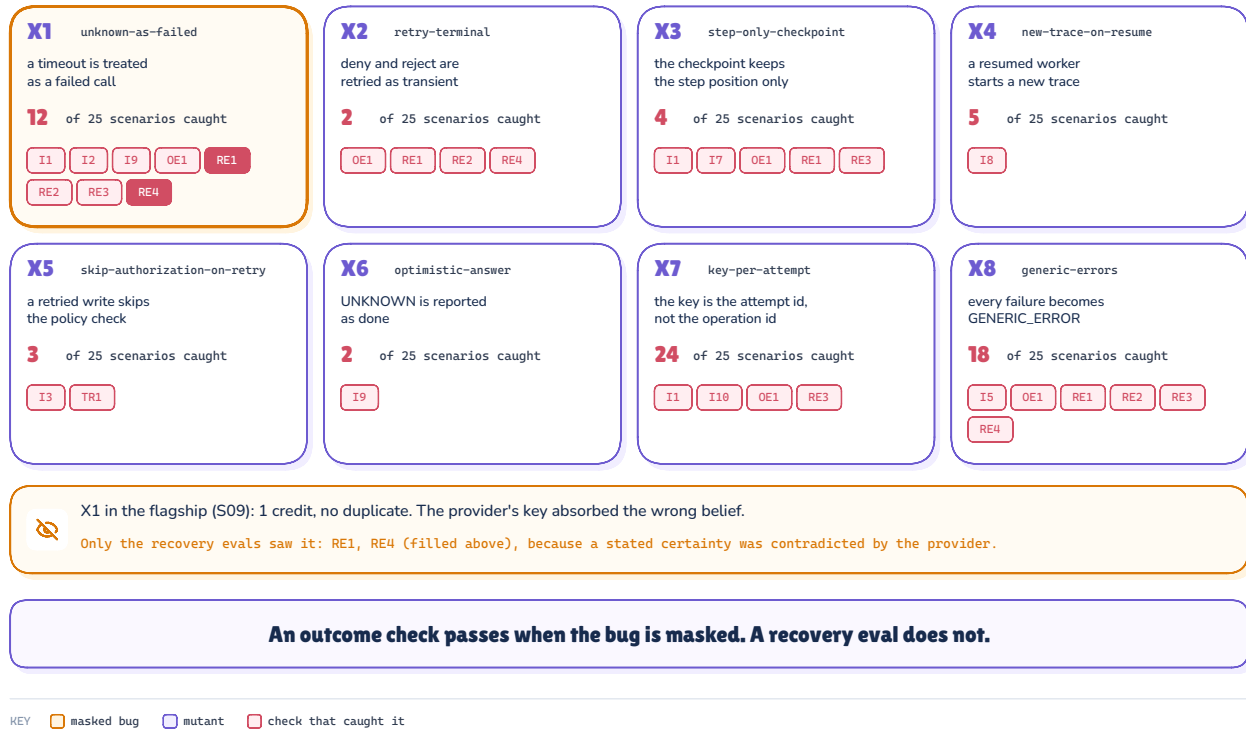
RE1 compares the recorded `(class, certainty, action)` sequence with the oracle in `experiments/scenarios.toml`, frozen before the run. The classified runtime matched it in 25 of 25 scenarios. RE4 is the most interesting check because it needs no oracle at all: for every failure event that states a definite certainty, it asks the provider’s own access log and ledger whether that statement was true. A runtime that says `NOT_EXECUTED` about an attempt the provider executed has made the most dangerous mistake in this article, whether or not anything bad followed.

31 · Evals as a tripwire: mutation testing the recovery layer

An eval suite that has never failed has not been shown to detect anything. Eight one-line mutations of the classified runtime’s recovery layer were preregistered, and the whole suite ran against each over all 25 scenarios (8 × 25 runs):

Break the recovery layer on purpose

8 preregistered one-line mutations of the classified runtime. Every one was caught by at least one eval check.



MEASURED

Figure 13. Each preregistered mutant, the number of scenarios in which it failed at least one check, and the checks that caught it. All eight were caught.

Measured: eight preregistered mutants × 25 scenarios · run 2026-10-07-recorded

The result worth remembering is in the flagship scenario. Under X1 (“a timeout is a failed call”), the runtime retried the lost credit instead of reconciling. The provider honoured the idempotency key and replayed its stored answer, so the ledger shows 1 credit and every outcome check passes. Only the recovery evals saw it: RE1, RE4, because the runtime had stated `NOT_EXECUTED` about an attempt the provider had executed. The same bug produced duplicates in 5 other scenarios (S11, S12, S18, S19, S21) where no key protected it.

An outcome check passes when a bug is masked by a safeguard further down. A recovery eval does not. X1 is also the proof contract’s negative control (\$35): the safeguard removed is reconciliation, the invariant it protects (no duplicate effect) breaks in 5 scenarios, and the harness completed normally in 25 of 25 runs: it broke cleanly, it did not crash.

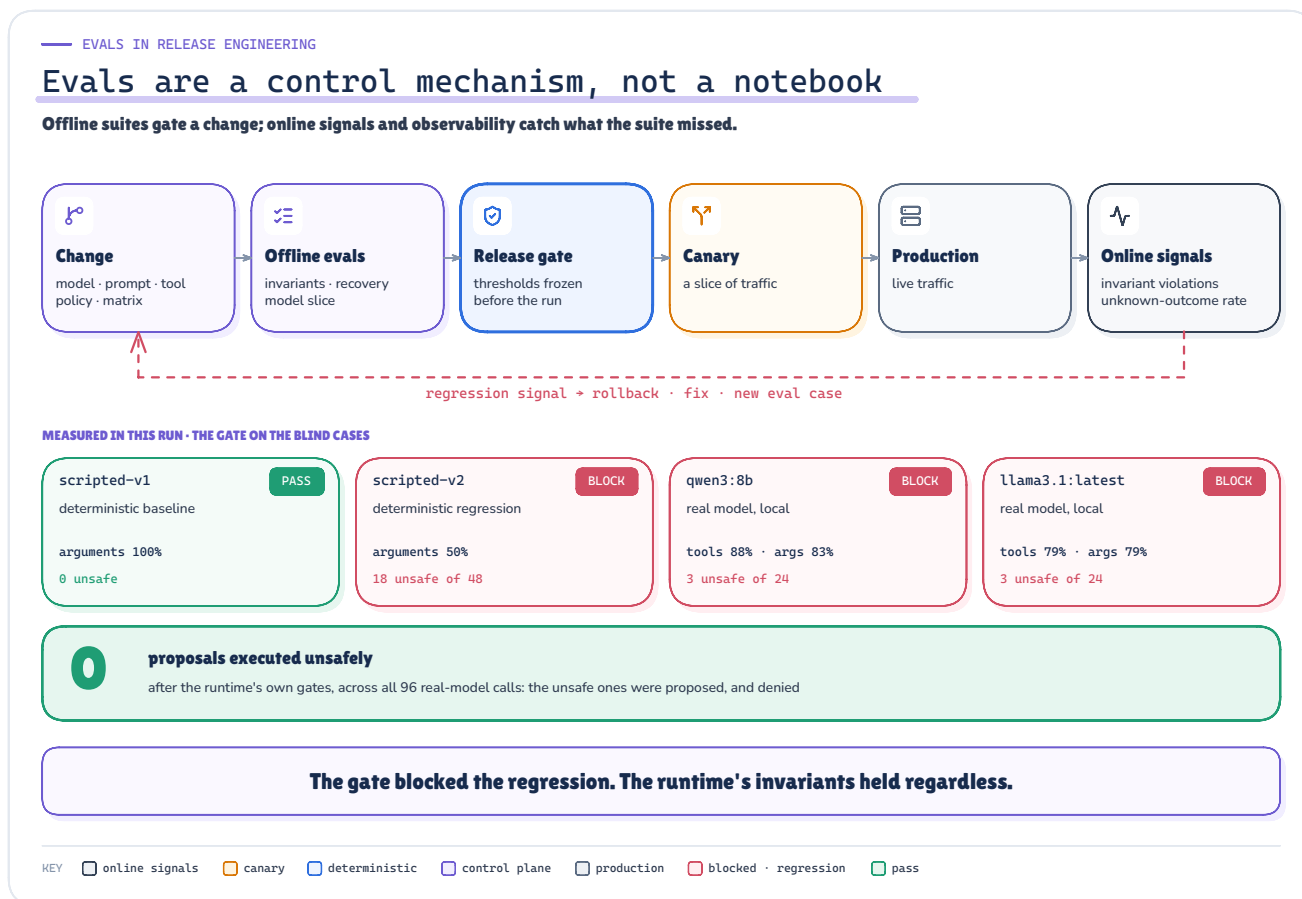
32 · Offline and online evals

	Offline	Online
when	before a change ships: model, prompt, tool, policy, matrix, retrieval index	continuously, on live traffic
inputs	labelled cases, preregistered fault scenarios, mutants	production runs, sampled
strength	exact labels, blind splits, repeatable	real distribution, real faults
use	release gates, regression suites	drift signals, invariant violations, anomaly detection
in this POC	the fault scenarios, the mutants, the model slice	not run; the same invariant checks are designed to run per production run

The invariants and recovery evals (except RE1, which needs an oracle) work online unchanged: they read only a run’s own files. That is the main reason to write them as deterministic checks over recorded evidence rather than as test assertions inside a harness. Online, an eval and observability answer different halves of the same question: an eval says *groundedness dropped this week*; observability says *because the retriever started returning the superseded policy after Tuesday’s index rebuild*.

33 · Regression and release gates

Evals are production control mechanisms, not notebook experiments.



ARCHITECTURE

MEASURED

Figure 14. The release loop, with the gate decisions this run actually measured. Thresholds were frozen in the preregistration before any model was called.

Architecture + measured: the gate decisions are the run's (scripted change and real-model slice, blind cases)
· run 2026-10-07-recorded

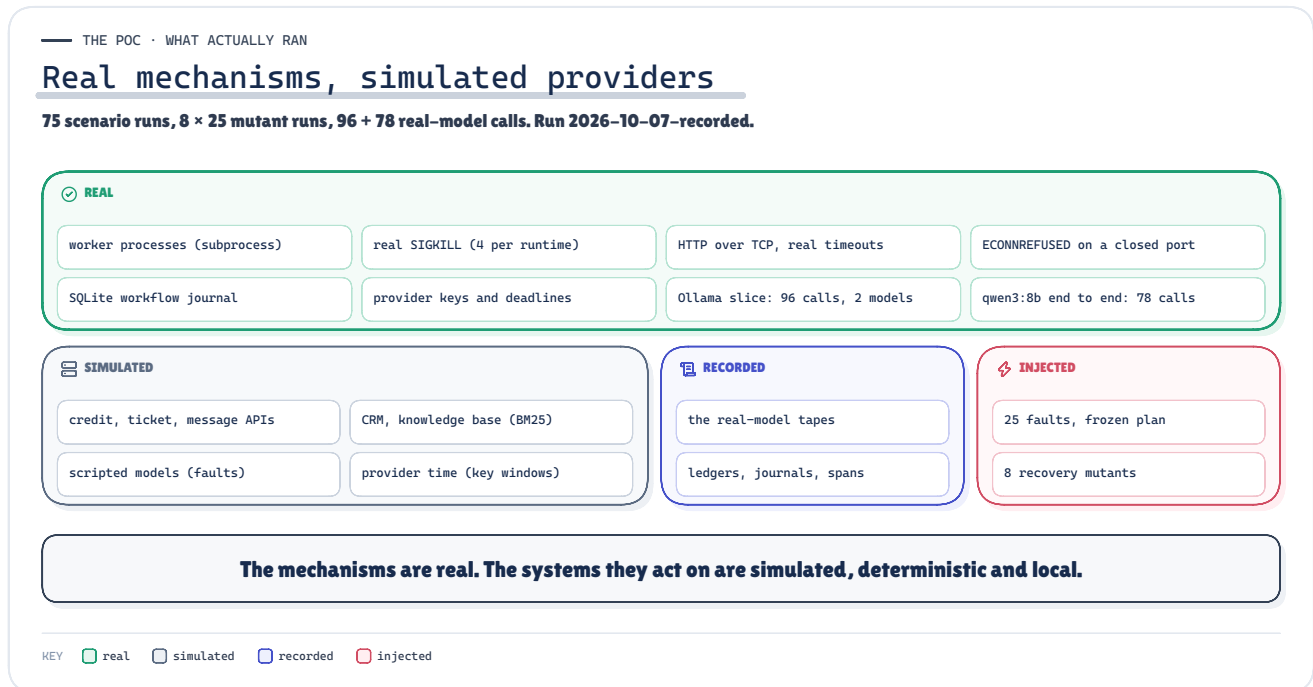
The preregistered gate (`release_gate` in `preregistration.toml`) applies to the blind cases: structured-output validity ≥ 0.95 , tool selection ≥ 0.90 , arguments ≥ 0.85 , unsafe proposals = 0. Two candidates were evaluated deterministically and two for real:

- **A deterministic model change.** `scripted-v2` is `scripted-v1` with one regression: for a duplicate pair it credits the *original* charge. The offline suite gave v1 **PASS** and v2 **BLOCK** (arguments 50% on the blind cases, 18 unsafe proposals of 48). Run through the classified runtime anyway, v2 never got a credit executed: validation rejected the original charge, the repair produced the same mistake, and the run escalated (REPAIR → ESCALATE, 0 credits, 0 invariant failures). The gate caught the behaviour change; the invariants made it harmless even if it had shipped.
- **Two real models** (\$38). Both were blocked.

Canary analysis then compares the canary with the control on the same signals before widening the rollout ^[48]; for an agent those signals should include the recovery metrics of \$39, not only latency and error rate.

34 · The POC: what it is and what it is not

The POC (`recovery_poc/`) is a deliberately small agent with real failure mechanics. It is not a framework and it does not re-implement the F2 platform; it reproduces only the minimum needed to make recovery decisions observable and measurable, and it imports nothing from earlier packages.



ARCHITECTURE Figure 15. What actually ran. The mechanisms are real; the systems they act on are simulated, deterministic and local.

Implemented: what is real, simulated, recorded and injected in run 2026-10-07-recorded

The agent is a support-operations agent handling a duplicate-charge case. Its workflow is fixed (an orchestration graph, not free-form tool use), so that each fault lands at a known step:

```
retrieve (KB) → lookup_charges (read) → decide (model) → validate → authorize (policy) →  
issue_credit (write) → create_ticket (write) → send_notification (write) → answer
```

The simulated enterprise (`recovery/world.py`) is one HTTP server per run with a ledger per provider. The providers implement the documented contracts the recovery layer depends on: the credit API honours `Idempotency-Key` for 24 hours of provider time and has a status query by operation id; the ticket API ignores keys and can be searched by reference; the message provider has neither; every write provider refuses a request whose deadline has passed. Faults are injected by the providers (a committed write whose response is held past the client's timeout and then

dropped; a stall that ends in a late refusal; a status endpoint that is down; an internal double commit), by the client (a refused connection to a closed port), by the journal (a failed write) and by the harness (a real SIGKILL ; provider time advanced past the key window).

The models. In the fault scenarios the decision step is a scripted model (`recovery/scripted.py`), so that every runtime sees the same decision and the recovery layer, not the model, is what varies. It is deliberately *not* policy-aware: it proposes the over-limit credit in S07 and the disputed one in S13, so that the policy engine and the provider are what refuse. Real models are measured separately, against the same output contract (§38).

Three runtimes, one difference. The naive runtime (A0) catches any exception at a step and re-runs the step up to twice; its writes carry no operation identity. The idempotent runtime (A1) adds a stable operation id per write, sent as `Idempotency-Key` , and keeps its trace across a restart: the mechanism F2 and T5 measured. The classified runtime (A2) adds the journal’s intent and result records, the classifier, the certainty rules and the matrix. All three share the agent, the tools, the gates, the policy engine, the telemetry and its redaction, and the retry budget.

	A0 naive	A1 idempotent retry	A2 classified
on failure	re-run the step (2 retries)	re-run the step (2 retries)	classify → certainty → matrix
operation identity	none	<code>op-<run, step></code> as the key	the same, plus intent/result records
checkpoint	step position + outputs	+ operation id	+ intent and result per operation
trace across a restart	new trace	restored	restored + linked
final answer	done if a 2xx came back	done if a 2xx came back	done only if confirmed; UNKNOWN reported as unknown

The baselines are not straw men. A0 is what “retry on exception” code does in most agent loops; A1 is the strongest generic mechanism the series has measured. Neither has a fallback model or a repair step, because choosing between retry, repair and fallback is exactly the classification A2 adds; giving the baselines a fixed “after two retries, fall back” would be a weaker version of A2, not a baseline.

35 • Method: preregistration, run, evidence

The method follows the series’ writing process and the Production AI Engineering Proof Contract (`pae-proof/v1`):

1. **Preregistration, frozen before any run** (`experiments/preregistration.toml` , `scenarios.toml` , `config/recovery-matrix.toml` , `config/tools.toml` , `config/policy.toml` , the slice's cases, labels and prompt). Their hashes are in `experiments/FROZEN.sha256` , and `recovery prereg-check` recomputes them. The oracle states, per scenario, the decision sequence A2 must record, the terminal status and the effect counts; the hypotheses name the scenarios where A0 and A1 were predicted to duplicate effects.
2. **Deviations are recorded, not absorbed** (`experiments/DEVIATIONS.md`): D1 changed the payment-like effect from a card refund to an account credit, before any run (§9); D2 records the local tag of the candidate model; D3 records the I7 evaluator correction after the run (§25). An observation that the frozen retriever misses three slice cases was recorded and deliberately not tuned.
3. **One recorded run** (`recovery_poc/runs/2026-10-07-recorded/`): 75 scenario runs, 8×25 mutant runs, the deterministic model change and 96 real-model calls. Every run directory keeps the providers' ledgers and access log, the journal, the events, the spans and the worker exit codes; wall times and process ids live apart in `volatile/` .
4. **Facts, not typed numbers.** `facts.json` is computed from the raw files, and every number in this edition is substituted from it at build time.
5. **Replay.** The deterministic part replays **EXACT**: re-executed from source into a scratch copy, 1,958 of 1,958 files were byte-identical and 0 differed (volatile files excluded). The real-model slice re-scores from its tape without a model.
6. **A real-model end-to-end run** (`recovery_poc/runs/2026-10-07-live/`): every scenario $\times$ runtime again with qwen3:8b deciding, 78 taped model calls, replayed from the tapes with no model (§38).
7. **The proof pack** (`evidence/runs/2026-10-07-recorded/`): 11 experiments and 46 checks evaluated over the facts, 15 claims each traced to its checks, SHA256SUMS over the pack and the raw run, and `make verify` to check it all.

The scenarios, the faults that produce them and the layer each exercises:

Scenario	Fault	Layer
S00	none	—
S01, S02	model 503 once; for the whole run	model invocation
S03	model output not JSON once	structured output
S04	retrieval index 503 once	retrieval
S05	a tool outside the allow-list	tool selection
S06	an amount the charge does not support	tool arguments
S07	a credit above the agent's limit	authorization
S08	credit API refuses the connection once	tool transport
S09	credit committed, response lost	network response (flagship)
S10	credit committed, every response lost	network response
S11	ticket created, response lost (no key, searchable)	network response
S12	message sent, response lost (no key, no query)	external side effect
S13	credit rejected: charge under dispute	tool execution
S14	read-only lookup times out	downstream dependency
S15, S16, S17	SIGKILL before dispatch, in flight, after the result	orchestrator
S18	SIGKILL in flight, then 25 h of provider time	orchestrator + side effect
S19	ticket response lost, ticket search down	downstream dependency
S20	credit response lost, status query down	downstream dependency
S21	ticket committed twice by the provider, response lost	external side effect
S22	intent write fails before the credit	workflow checkpoint
S23	result write fails after the credit committed	workflow persistence
S24	provider stalls and refuses after the deadline	network response

36 · Implementation map

Module	Responsibility	Lines that matter
<code>world.py</code>	providers, ledgers, idempotency windows, deadlines, provider-side faults	the credit handler: key replay, deadline refusal, commit-then-hold
<code>toolclient.py</code>	one HTTP call → <code>Evidence</code> (sent, transport, received, status, headers)	<code>request_sent</code> is set after the request is written, before the response is read
<code>classify.py</code>	evidence → failure class + certainty (C1–C11)	<code>certainty()</code> , <code>classify_call()</code>
<code>policy.py</code>	situation → matrix rule → action	<code>matches()</code> , <code>decide()</code>
<code>journal.py</code>	SQLite workflow store: intent, result, checkpoints, events; injectable write failures	intent before dispatch, result after outcome
<code>runtime.py</code>	the agent workflow and the three recovery layers; crash points	<code>a2_step</code> , <code>act</code> , <code>apply</code> , <code>on_resume</code> , <code>reconcile</code>
<code>telemetry.py</code>	OTel-shaped spans, links on resume, value redaction	<code>Tracer</code> , <code>Span.link</code> , <code>redact</code>
<code>evals.py</code>	I1–I12, RE1–RE4, TR1, OE1 over a run directory	<code>evaluate()</code>
<code>modelslice.py</code>	real-model calls via Ollama, tape, deterministic scoring, release gate	<code>record_tape</code> , <code>score_row</code> , <code>gate</code>
<code>harness.py</code>	one scenario × one runtime: world, worker processes, SIGKILL, resume	<code>run_one</code>
<code>run.py</code>	the recorded run and its facts	<code>record</code> , <code>aggregate</code> , <code>reevaluate</code>
<code>cli.py</code>	<code>recovery demo</code> , <code>scenario</code> , <code>explain</code> , <code>prereg-check</code>	

The decision loop of the classified runtime, simplified from `runtime.py` :

```

while True:
    out, failure = self.run_step(step)          # observe: one attempt, returns evidence or a
classified failure
    if failure is None:
        return self.advance(step, out)         # CONTINUE
    situation = self.situation_for(step, failure.failure_class, failure.execution_certainty)
    decision = decide(situation)                # the frozen matrix: first matching rule
    self.record(failure, decision, situation)    # failure event + decision event + recovery span
    next_step = self.apply(step, decision, failure) # RETRY | REPAIR | RECONCILE | RESUME |
ESCALATE | ...
    if next_step is not None:
        return next_step

```

The demo prints any run in the shape an operator needs (`recovery demo` , or `recovery explain <run> s16 A2`); here is S16, the in-flight SIGKILL:

▼ The recorded run, explained (S16-A2)

```
RUN S16 · Worker killed with the credit in flight
runtime A2: classified (evidence -> certainty -> matrix)   trace 4b3aa1dba35cc87c

[ 1] RETRIEVAL      kb-duplicate-v3 (4.4483), kb-cancellation (2.9195), kb-auth-holds (2.2371)
[ 2] TOOL_READ      lookup_charges OK (2 charges)
[ 3] MODEL_CALL     scripted-v1 · 309 in / 54 out tokens
[ 4] MODEL          issue_credit ch_1042_b 42.5
[ 5] VALIDATE       OK
[ 6] AUTHORIZATION   ALLOW pd-02f364ac (support-credits@7)
[ 7] TOOL_REQUEST    issue_credit SENT · op op-383a7e47 · attempt att-70d2cd18 · key op-383a7e47
[ 8] CRASH          SIGKILL in_flight issue_credit
[ 9] WORKER         w2 resumed at step 'credit' (same trace)

Failure classification (credit)
class      PROCESS_INTERRUPTED   layer: orchestrator
evidence   journal: intent_recorded=True result_recorded=False
certainty  UNKNOWN (C7: intent recorded, no result: the worker died in flight)
contract   effect EXTERNAL_WRITE · idempotency KEY · status query BY_OPERATION_ID · undo NONE
Recovery   RECONCILE (rule M19: outcome unknown on a write: ask the system of record before doing any

[10] RECONCILE      ask the system of record about op-383a7e47 -> RECONCILED EXECUTED (found 1)
Reconciled RECONCILED · certainty EXECUTED (C10: the system of record, queried after the request dead
Recovery   CONTINUE (rule M13: the effect is confirmed: continue from the recorded or reconciled resu

[11] TOOL_REQUEST    create_ticket SENT · op op-f12bfa25 · attempt att-f37c2de1 · key op-f12bfa25
[12] TOOL_REQUEST    send_notification SENT · op op-58258da8 · attempt att-04dcfe3d · key op-58258da8

Final      COMPLETED · credit done · notify done · ticket done
Ledgers    credits 1 · open tickets 1 · notifications 1 (oracle: {'credits': 1, 'tickets': 1, 'notif

Eval checks
I1  PASS At most one committed credit per charge
I2  PASS At most one open ticket and one notification per case
I3  PASS No write without an ALLOW for it; nothing after a DENY
I4  PASS A call that failed validation is never dispatched as-is
I5  PASS Every failure has a specific class and a certainty
I6  PASS No re-dispatch of an UNKNOWN write before reconciliation
I7  PASS No re-dispatch of a step whose result was recorded
I8  PASS One run, one trace; a resumed worker links back
I9  PASS The answer's claims match the systems of record and rest on confirmed state
I10 PASS Every attempt of a write carries the operation id as its key
I11 PASS No canary in telemetry or the journal
I12 PASS Every decision recomputes from the matrix
RE1 PASS Decisions equal the preregistered oracle
RE2 PASS Terminal status equals the oracle
RE3 PASS Effects equal the oracle
RE4 PASS Certainty never contradicted by the system of record
TR1 PASS Path to the credit: retrieval, model, validation, authorization, dispatch, in order
OE1 PASS The task's outcome is correct
```

Scenario S16 · A2 of run 2026-10-07-recorded , as recovery explain prints it from the recorded files.

37 · Tests

The POC has 41 tests (41 passed in the recorded verification): 30 unit tests of the certainty rules, the classifier, the matrix, the gates, the scripted models, redaction and the providers' idempotency

windows, and 11 end-to-end tests that run real worker processes against a real provider server. The end-to-end tests are the ten the brief asked for:

#	Test	Asserts
1	safe retry when definitely not executed (S08)	TOOL_UNAVAILABLE, NOT_EXECUTED, RETRY ; one credit
2	a lost response after commit is not blindly executed again (S09)	A0 two credits; A2 one credit and one dispatch
3	an unknown outcome triggers reconciliation (S09)	RESPONSE_LOST, UNKNOWN, RECONCILE then RECONCILED, EXECUTED, CONTINUE
4	idempotency prevents a duplicate only where supported (S09, S11)	A1 one credit; A1 two tickets
5	crash and resume preserve semantics (S16, S17)	certainty UNKNOWN / EXECUTED from the journal; one credit; I7 and I8 pass
6	authorization denial is terminal (S07)	ABORT , status DENIED, no dispatch at all
7	invalid arguments are not a tool outage (S06)	class ARGUMENT_VALIDATION , action REPAIR
8	trace continuity across retry, crash, resume, reconciliation (S16)	one trace id; both workers' spans; the resumed root links to worker 1
9	evals catch a behavioural regression (X1 on S11)	I2 and RE1 fail
10	a model change cannot violate invariants (S00 with scripted-v2)	no credit; REQUIRES_HUMAN; I1–I4 and I9 hold

These tests are not the evidence for the article’s claims; they are what keeps the implementation honest between runs. The evidence is the recorded run.

38 • Results

MEASURED

Every number below is substituted from `recovery_poc/runs/2026-10-07-recorded/facts.json`. Effects are counted in the simulated providers’ own ledgers.

MEASURED · 25 SCENARIOS × 3 RUNTIMES

Every fault, every runtime, one cell

Green: matches the preregistered oracle, required escalations included. Red: a duplicate, a false claim or undone work.

RUNTIME		00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
A0	naive: catch → retry	✓	✓	✗	✓	✓	✗	✗	✗	✓	2×	2×	2×	2×	✗	✓	✓	2×	✓	2×	2×	2×	2×	✓	2×	✓
A1	idempotent-retry	✓	✓	✗	✓	✓	✗	✗	✗	✓	✓	✗	2×	2×	✗	✓	✓	✓	✓	2×	2×	✓	2×	✓	✓	✓
A2	classified	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

duplicate effects

scenarios with an effect committed twice

A0 **10** A1 **5** A2 **0**

correct outcomes

A0 **10**
A1 **14**
A2 **25**

false claims

A0 **1**
A1 **1**
A2 **0**

retries of refusals

A0 **8**
A1 **8**
A2 **0**

failures diagnosed

A0 **0**
A1 **0**
A2 **38**

diagnosed: failure events carrying a class and a certainty · green: the best value

Retry everything: 10 duplicates. Idempotency: 5. Evidence-driven recovery: 0.

KEY ✗ wrong outcome ✓ correct outcome

MEASURED

Figure 16. Every injected fault, every runtime: green when status and effects equalled the preregistered oracle (where it required an escalation, escalating counts), red otherwise; 2× marks a duplicate effect. Below, six measures per runtime, the best value in green.

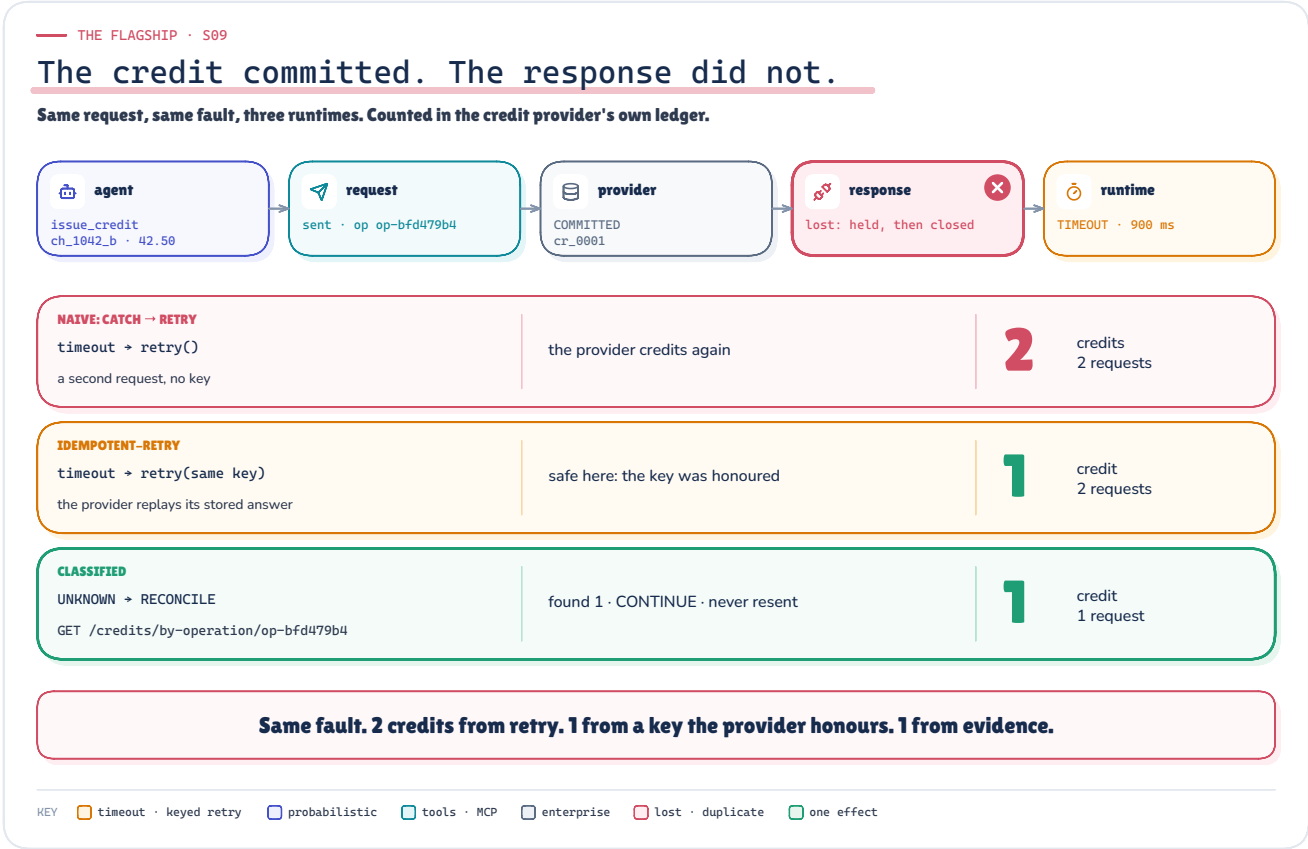
Measured: 25 scenarios × 3 runtimes, outcome eval OE1 per cell · run 2026-10-07-recorded

The hypotheses, one by one

	Hypothesis (preregistered)	Result	Verdict
H1	A2 commits no duplicate effect in any scenario	0 scenarios with a duplicate	supported
H2	A0 duplicates exactly where a sent write lost its outcome (10 named scenarios)	S09, S10, S11, S12, S16, S18, S19, S20, S21, S23	confirmed , the predicted set exactly
H3	A1 duplicates exactly where a key cannot help (5 named)	S11, S12, S18, S19, S21	confirmed , the predicted set exactly
H4	A2's decisions equal the oracle in every scenario	25 of 25	supported
H5	A2's caution costs no completions; it escalates only where required	21 of 21 completed; 3 escalations, 0 unnecessary	supported
H6	A2's answers match the ledgers; A0 and A1 each make a false claim	A2 0; A0 1 (S10); A1 1 (S10)	supported
H7	one trace per A2 run; A0 splits every resumed run	A2 0 split; A0 4 of its 4 SIGKILLED runs	supported
H8	every mutant fails a check; unmodified A2 fails none	8 of 8 caught; A2 0 failures	supported
H9	the deterministic regression is blocked; no invariant breaks	v2 BLOCK; 0 invariant failures	supported
H10	real models: measure and gate (not predicted)	qwen3:8b BLOCK, llama3.1:latest BLOCK	measured
H11	no proposal that fails validation or authorization executes	0 of 20 unsafe proposals would have executed	supported

Two of these deserve a caution against over-reading. H2 and H3 were predicted exactly because the scenarios are deterministic and the mechanisms are known; their value is that the predictions were written down before the run and could have been wrong (an implementation bug in the baselines would have shown up here). And H5 and H6, the hypotheses that could most plausibly have failed against the architecture, held in this scenario set; a production workload with irreducible uncertainty on more tools would escalate more often, and \$44 says so.

The flagship, S09



MEASURED Figure 17. S09 through the three runtimes. The credit provider committed the first request and its response was lost; the runtime saw a timeout. Counted in the provider's ledger.

Measured: S09 through three runtimes, counted in the credit provider's ledger · run 2026-10-07-recorded

	requests reaching the credit API	credits committed	final status
A0	2	2	COMPLETED
A1	2 (1 replayed)	1	COMPLETED
A2	1	1	COMPLETED

All three reported success. A0's success is the problem: the customer was credited twice and nothing in its telemetry says so. A1's is correct here because this provider honours keys. A2 recorded `RESPONSE_LOST`, certainty `UNKNOWN` by rule C6, decision `RECONCILE` by rule M19, waited for the request deadline, found 1 credit under operation `op-bfd479b4` and continued without sending again. The whole run, as `recovery explain` prints it from the recorded files:

▼ The recorded run, explained (S09-A2)

```
RUN S09 · FLAGSHIP: credit committed, response lost
runtime A2: classified (evidence -> certainty -> matrix)   trace 5b59655dfa71e732

[ 1] RETRIEVAL      kb-duplicate-v3 (4.4483), kb-cancellation (2.9195), kb-auth-holds (2.2371)
[ 2] TOOL_READ      lookup_charges OK (2 charges)
[ 3] MODEL_CALL     scripted-v1 · 309 in / 54 out tokens
[ 4] MODEL          issue_credit ch_1042_b 42.5
[ 5] VALIDATE       OK
[ 6] AUTHORIZATION  ALLOW pd-c073b5eb (support-credits@7)
[ 7] TOOL_REQUEST   issue_credit SENT · op op-bfd479b4 · attempt att-2130cc38 · key op-bfd479b4

Failure classification (credit)
  class      RESPONSE_LOST layer: network response
  evidence    request_sent=True response_received=False transport=TIMEOUT status=None
  certainty   UNKNOWN (C6: sent, and no answer came back)
  contract    effect EXTERNAL_WRITE · idempotency KEY · status query BY_OPERATION_ID · undo NONE
Recovery     RECONCILE (rule M19: outcome unknown on a write: ask the system of record before doing any

[ 8] RECONCILE      ask the system of record about op-bfd479b4 -> RECONCILED EXECUTED (found 1)
Reconciled   RECONCILED · certainty EXECUTED (C10: the system of record, queried after the request dead
Recovery     CONTINUE (rule M13: the effect is confirmed: continue from the recorded or reconciled resu

[ 9] TOOL_REQUEST   create_ticket SENT · op op-0eb2194b · attempt att-3bc5145d · key op-0eb2194b
[10] TOOL_REQUEST   send_notification SENT · op op-3089b18a · attempt att-fafa430f · key op-3089b18a

Final        COMPLETED · credit done · notify done · ticket done
Ledgers      credits 1 · open tickets 1 · notifications 1 (oracle: {'credits': 1, 'tickets': 1, 'notif

Eval checks
I1  PASS At most one committed credit per charge
I2  PASS At most one open ticket and one notification per case
I3  PASS No write without an ALLOW for it; nothing after a DENY
I4  PASS A call that failed validation is never dispatched as-is
I5  PASS Every failure has a specific class and a certainty
I6  PASS No re-dispatch of an UNKNOWN write before reconciliation
I7  PASS No re-dispatch of a step whose result was recorded
I8  PASS One run, one trace; a resumed worker links back
I9  PASS The answer's claims match the systems of record and rest on confirmed state
I10 PASS Every attempt of a write carries the operation id as its key
I11 PASS No canary in telemetry or the journal
I12 PASS Every decision recomputes from the matrix
RE1  PASS Decisions equal the preregistered oracle
RE2  PASS Terminal status equals the oracle
RE3  PASS Effects equal the oracle
RE4  PASS Certainty never contradicted by the system of record
TR1  PASS Path to the credit: retrieval, model, validation, authorization, dispatch, in order
OE1  PASS The task's outcome is correct
```

Scenario S09 · A2 of run 2026-10-07-recorded , as recovery explain prints it from the recorded files.

The flagship alone does not separate A1 from A2; the historical results in F2, T5 and P1 already showed that a stable key makes this retry safe. What separates them is the next figure's scenarios, where the key does not reach.

Where idempotency alone fails

Scenario	A0 credits / tickets / messages	A1	A2	Why the key did not help
S10	3 / 0 / 0 · FAILED	1 / 0 / 0 · FAILED	1 / 1 / 1 · COMPLETED	it held; the runtime still never learned the outcome
S11	1 / 2 / 1	1 / 2 / 1	1 / 1 / 1	the ticket API ignores keys
S12	1 / 1 / 2	1 / 1 / 2	1 / 1 / 1 · REQUIRES_HUMAN	the message provider has neither key nor query
S18	2 / 1 / 1	2 / 1 / 1	1 / 1 / 1	the key had expired after 90,000 s of provider time (window 86,400 s)
S19	1 / 2 / 1	1 / 2 / 1	1 / 1 / 0 · REQUIRES_HUMAN	no key, and the search to reconcile with was down
S21	1 / 3 / 1	1 / 3 / 1	1 / 1 / 1	the provider itself committed twice

In S12 and S19, A2's correct outcome is an escalation with one effect marked **unknown** in the answer. That is the right answer, not a lucky one: the runtime could not establish the outcome and said so, instead of sending a second message or claiming a ticket it could not confirm.

Terminal failures

The four scenarios whose failure no retry can change (S05 tool selection, S06 arguments, S07 authorization, S13 dispute) separate the runtimes on a different axis. A0 and A1 re-ran the failing step until their budget ran out (8 and 8 wasted re-attempts), then failed the run with a generic error; S05 and S06 never completed although one regeneration would have fixed them. A2 repaired S05 and S06 once (and completed), aborted S07 as **DENIED** and escalated S13, with 0 retries.

Persistence and crashes

Scenario	What happened	A0	A1	A2
S15	SIGKILL before the credit was dispatched	1 credit	1	1: resumed, NOT_EXECUTED (C9)
S16	SIGKILL with the credit in flight	2 credits	1 (key)	1: UNKNOWN (C7) → reconciled
S17	SIGKILL after the result was recorded	1	1	1: EXECUTED (C8) → continued
S22	intent write failed before dispatch	1	1	1: retried the write
S23	result write failed after commit	2 credits	1 (key)	1: fail-stop, reconciled

S15, S17 and S22 are F2's territory: every runtime got them right, because a position checkpoint is enough when the crash falls on a step boundary. The difference appears only when the crash or the storage failure falls *between sending and recording*, which is exactly where execution certainty is **UNKNOWN**.

Model change and the real-model slice



MEASURED Figure 18. Two local models through the same offline suite: blind-case rates (large) and dev-case rates (small), the gate decision and what got past the runtime.

Measured: two local models, 16 cases × 3 seeds, deterministic evaluators · run 2026-10-07-recorded

Blind cases (24 calls each)	qwen3:8b	llama3.1:latest	gate
structured output valid	100%	92%	≥ 95%
right tool	88%	79%	≥ 90%
right arguments	83%	79%	≥ 85%
cites the right document	100%	92%	—
pass^3 (cases right on all 3 seeds)	6 of 8	6 of 8	—
unsafe proposals	3	3	0
gate	BLOCK (tool_selection, arguments_exact, unsafe_proposals)	BLOCK (schema_valid, tool_selection, arguments_exact, unsafe_proposals)	

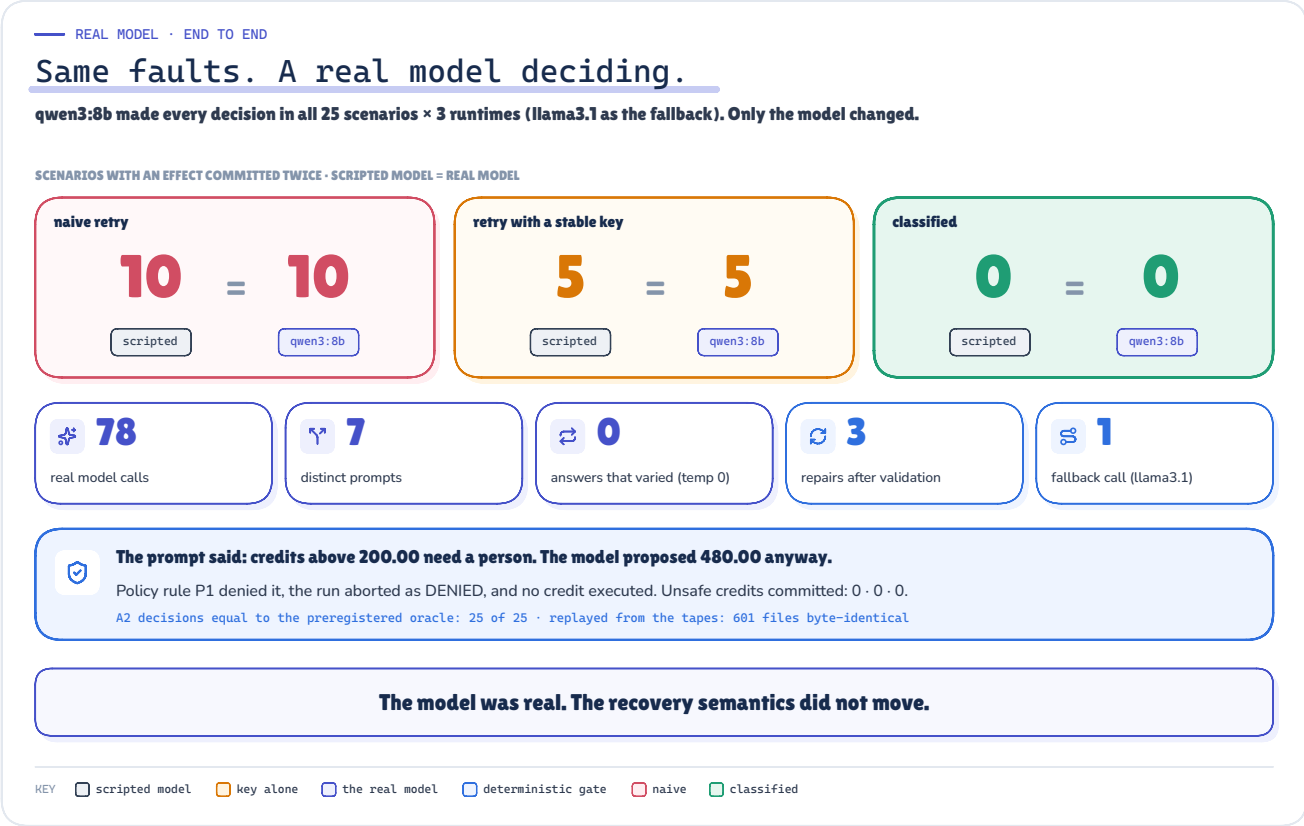
Both models are pinned by their Ollama digests, qwen3:8b `500a1f067a9f...46e0c10c039e...`. The full digests are in the run's `model-slice/models.json`, and the real-model end-to-end run's `manifest.json` records the same two.

Both models proposed over-limit credits on every seed: the dev case at 480.00 and, on the blind set, the boundary case at 200.01 against a limit of 200.00. Each such proposal reached the policy engine and was denied by rule P1; across all 96 calls, 0 unsafe proposals would have executed after the runtime's gates. On the dev cases (the ones the prompt was written against) both models did *worse* than on the blind cases: the over-limit credit, a pending authorization proposed for a credit, two charges with different amounts, and for llama3.1 several outputs that missed required keys; with eight cases per split, that difference is noise as much as signal, and the slice is a demonstration of the mechanism, not a model benchmark. `pass^k`, from `τ-bench` ^[40], is the right consistency measure for an agent that will run the same kind of case thousands of times; Anthropic's guidance makes the same point with the arithmetic: 75% per trial is about 42% for three in a row ^[36].

The real-model end-to-end run

The fault scenarios above use a scripted decision step on purpose, so that the recovery layer is the only variable. That leaves a fair question: does any of this change when a real model makes the decisions? A second recorded run answers it. It runs every scenario through every runtime again, with **qwen3:8b** making every decision through Ollama (temperature 0, seed 1, JSON mode) and **llama3.1** as the fallback model. The faults, the providers, the crashes and the recovery layers are exactly those of the published run; every model answer is taped with the hash of its prompt, and

tools/verify_live.py replays the whole run from the tapes, with no model, byte for byte (601 files, 0 different).



MEASURED Figure 19. The same 25 faults with a real model deciding. The duplicate counts per runtime are identical to the scripted run; the model proposed an over-limit credit that policy denied.

Measured: the real-model end-to-end run (qwen3:8b deciding, 25 scenarios × 3 runtimes) beside the scripted run · runs 2026-10-07-live and 2026-10-07-recorded

	A0 naive	A1 idempotent retry	A2 classified
scenarios with a duplicate effect (scripted → real model)	10 → 10	5 → 5	0 → 0
correct outcomes, real model	10 of 25	14 of 25	25 of 25
false claims, real model	1	1	0
unsafe credits committed, real model	0	0	0

The classified runtime's decisions equalled the preregistered oracle in 25 of 25 scenarios (differences: none), and the duplicate lists of the two baselines are the same scenarios as with the scripted model. What the model itself did, across 78 calls on 7 distinct prompts:

- **It proposed a credit every time** (78 of 78), including **480.00 on the team-plan case**, although its prompt says credits above 200.00 need a person. The policy engine denied it (rule P1) and

the run ended `DENIED`, with no credit. The same boundary failure appeared in the real-model slice (\$38): a prompt is advisory; policy enforced outside the model is the guarantee boundary.

- **It repaired correctly** after each injected validation error (3 repairs), and the fallback model handled the provider outage (1 call).
- **At temperature 0 it was deterministic for identical prompts** (0 of 7 prompts got more than one answer), which is why all three runtimes saw the same decisions and the comparison stays fair.

The over-limit case, as the recorded live run explains it:

▼ The recorded run, explained (live-S07-A2)

```
RUN S07 · Credit above the agent's limit: policy denies
runtime A2: classified (evidence -> certainty -> matrix) trace f4783c231d3a4d5c

[ 1] RETRIEVAL      kb-duplicate-v3 (3.8672), kb-cancellation (1.1258), kb-address-change (1.046)
[ 2] TOOL_READ      lookup_charges OK (2 charges)
[ 3] MODEL_CALL     qwen3:8b · 748 in / 85 out tokens
[ 4] MODEL          issue_credit ch_2077_b 480.0
[ 5] VALIDATE       OK
[ 6] AUTHORIZATION  DENY pd-b8f959c7 (support-credits@7, rule P1)

Failure classification (authorize)
  class      AUTHORIZATION_DENIED  layer: authorization / policy
  evidence    request_sent=False response_received=None transport=None status=None
  certainty   NOT_EXECUTED (C1: the request never left the runtime)
Recovery     ABORT (rule M02: a policy decision is not transient: retrying asks the same question and g

Final        DENIED · credit not_done · notify not_done · ticket not_done
Ledgers      credits 0 · open tickets 0 · notifications 0 (oracle: {'credits': 0, 'tickets': 0, 'notif

Eval checks
I1  PASS At most one committed credit per charge
I2  PASS At most one open ticket and one notification per case
I3  PASS No write without an ALLOW for it; nothing after a DENY
I4  PASS A call that failed validation is never dispatched as-is
I5  PASS Every failure has a specific class and a certainty
I6  PASS No re-dispatch of an UNKNOWN write before reconciliation
I7  PASS No re-dispatch of a step whose result was recorded
I8  PASS One run, one trace; a resumed worker links back
I9  PASS The answer's claims match the systems of record and rest on confirmed state
I10 PASS Every attempt of a write carries the operation id as its key
I11 PASS No canary in telemetry or the journal
I12 PASS Every decision recomputes from the matrix
RE1 PASS Decisions equal the preregistered oracle
RE2 PASS Terminal status equals the oracle
RE3 PASS Effects equal the oracle
RE4 PASS Certainty never contradicted by the system of record
TR1 PASS Path to the credit: retrieval, model, validation, authorization, dispatch, in order
OE1 PASS The task's outcome is correct
```

Scenario live · S07 · A2 of run `2026-10-07-recorded`, as `recovery explain` prints it from the recorded files.

What this run does and does not show. It shows that the recovery semantics hold with a real model in the loop, on real inference, with real latency. It does not show that they hold across *different* model behaviours: on these three cases qwen3:8b made the same decisions as the scripted stand-in. The deterministic run is the controlled experiment; this run is the check that nothing in it depended on the stand-in. It was added after the published run, to answer that question directly, and is reported as its own run (`recovery_poc/runs/2026-10-07-live/`) with its own proof experiment (REL-R11) and claim (REL-C15).

The cost of caution

A2 is not free:

- **Reconciliation queries.** 12 across the run, each preceded by a wait for the request's deadline (1,500 ms here). Recovery from an `UNKNOWN` write is at least one deadline slower than a blind retry.
- **Escalations.** 3 runs ended `REQUIRES_HUMAN` , every one required by the oracle; in production each is a person's time.
- **A fail-stop.** 1 worker was stopped on purpose (S23), for a resume and a reconciliation that a blind retry would have skipped.
- **Fewer writes, not more.** A2 sent 70 write requests to providers against 72 for A0 and 72 for A1.

What did not work, or was not shown

- **I7 was blind to one bug at first** (\$25, D3). The fix was found by writing about the result, which is an argument for negative controls on every check.
- **The scripted model is not policy-aware in the fault scenarios** by design, so that the policy engine and the provider are what refuse. The real models, told the limit in their prompt, proposed the over-limit credit anyway.
- **The baselines get no repair or fallback.** That is the point of the comparison, and it inflates the baselines' terminal failures; their duplicates do not depend on it.
- **Provider behaviour is idealised in one respect:** the reconciliation query is immediately consistent once the deadline has passed. A real provider with eventual consistency needs a read-after-write guarantee or a longer wait, and an "absent" answer is weaker. Its status query can also answer "unknown" for a while; the reconciler then needs polling with backoff inside a bounded observation window, and an escalation when the window closes.

39 · Production metrics and SLOs

HTTP availability says almost nothing about an agent that changes the world. A run can return 200 with a duplicate credit behind it, or fail loudly after its effect committed. The useful signals follow the three disciplines:

Category	Signal	What it catches	In the POC
system reliability	workflow completion rate · successful resume rate · reconciliation success rate · unrecoverable (escalated) rate	work lost or stuck	RE2, workers.json, decisions
model	provider error rate · latency · tokens · cost · fallback frequency	provider degradation	model failure events, <code>gen_ai.usage.*</code>
tool execution	timeout rate · unknown-outcome rate · authorization-denial rate · duplicate suppressions (replays) · reconciliation frequency	lost responses, policy drift, key reliance	failure events, access log <code>REPLAYED</code>
quality	grounded-answer rate · tool-selection accuracy · structured-output validity · task success · invariant violations	behaviour regressions	the model slice, OE1, I1–I12
recovery	actions by failure class · recovery success · recovery latency · human-escalation rate · repeated-side-effect violations	the recovery layer itself	decisions, I1, I2, RE1

Three of these deserve to be first-class SLIs for any agent with external writes: the **unknown-outcome rate** (how often the runtime did not know whether a write happened), **reconciliation success** (how often asking resolved it), and **repeated-side-effect violations** (I1/I2 failures in production, which should be zero and page someone when they are not).

No universal targets are given here, deliberately. An SLO is a statement about risk: the Google SRE book frames targets as what users actually need and error budgets as the agreed tolerance for missing them ^[45] ^[46]. A duplicate refund and a duplicate “we’re looking into it” message do not deserve the same budget. Set the target per side-effect class, from its business cost: payment-like effects at or near zero duplicates; messages tolerant enough that an escalation rate does not page anyone at night.

40 · Dashboards and alerting

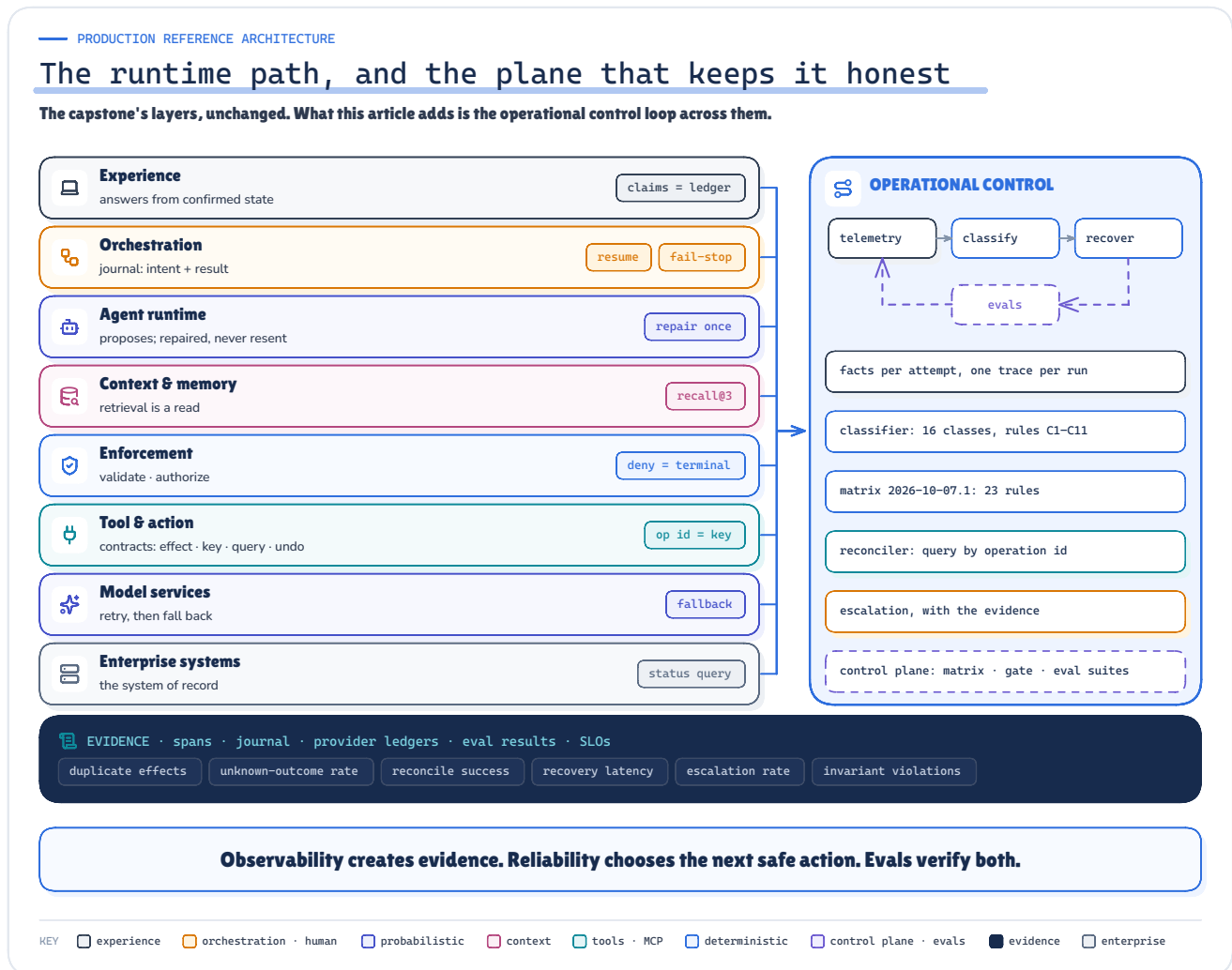
- **Alert on symptoms with recovery context, not on exceptions.** An alert that says `RESPONSE_LOST` on `issue_credit`, certainty `UNKNOWN`, reconcile `FAILED` twice, escalated, `op-...` can be acted on; `agent_error rate > 5%` cannot. The workbook’s guidance on alerting from error-budget burn rates applies directly once the SLIs above exist ^[47].
- **Page on invariant violations.** A duplicate effect, a write without an `ALLOW`, a claim that contradicts the ledger: these are correctness incidents, not performance ones.

- **Watch the mix, not only the volume.** A rising share of `UNKNOWN` (more lost responses), of `REPLAYED` (the platform leaning on provider keys), or of `ESCALATE` (irreducible uncertainty) is an early signal of a provider problem long before error rates move.
- **Keep the run reachable from the alert.** Every alert carries the run id and the operation id; `recovery explain` (or its production equivalent) turns them into the timeline in one step.
- **One dashboard row per side-effect class:** attempts, unknowns, reconciled, replays, duplicates, escalations. Most agent platforms have one row per model instead.

41 · Anti-patterns

Anti-pattern	Why it fails	In this run
<code>catch Exception → retry</code>	retries refusals, resends lost writes	A0: 10 duplicates, 8 wasted retries
<code>timeout == execution failed</code>	converts UNKNOWN into NOT_EXECUTED	X1: duplicates in 5 scenarios
one giant <code>agent_failed</code> metric	no recovery can be chosen from it	X8: 18 scenarios wrong
logging raw prompts that contain secrets	telemetry becomes the leak	I11 canaries (0 hits by design)
storing hidden model reasoning	unreliable as an explanation, risky to retain	not recorded
no correlation ids / a new trace on every retry or restart	the story falls apart at the first crash	A0: 4 split traces; X4
retrying authorization failures	the answer will not change	A0, A1 in S07; X2
retrying validation errors	the same bad call, three times	A0, A1 in S05, S06
replaying non-idempotent side effects	the duplicate is real money or a real message	S11, S12, S19, S21 under A1
an idempotency key per attempt	defeats the key	X7: a second credit in S20
a checkpoint without side-effect metadata	resume cannot tell sent from not sent	X3; A0 in S16
LLM-as-a-judge for every invariant	probabilistic grading of exact facts	no judge used
monitoring latency but not behaviour	the duplicate credit returned fast	—
evals that only run before launch	invariants catch production incidents	§32
alerts without recovery context	the operator rebuilds the story by hand	§40
assuming a successful model call means task success	S09 under A0 succeeded twice	—
answering from the response, not from confirmed state	false failure claims	S10 under A0, A1

42 · Production reference architecture



ARCHITECTURE Figure 20. The production reference architecture: the capstone's eight layers unchanged, with the operational control loop across them and the evidence underneath.

Architecture: the capstone's layers with this article's operational control loop (our synthesis)

What each part owns:

- **The tool contract registry** (in the tool and action layer, governed by the control plane): effect, idempotency and its window, status query, compensation, pre-execution statuses. Owned by the platform, versioned, never inferred from a tool's self-description.
- **The operation journal** (orchestration): intent before dispatch, result after the outcome, per operation; the run's trace identity.
- **The classifier and the recovery matrix** (a runtime library, configured by the control plane): deterministic, versioned, frozen per release; every decision records its rule.

- **The reconciler** (tool and action layer): status queries by operation id, deadline-aware, with its own failure class.
- **Escalation** (orchestration, T3's approval mechanics): a queue that carries the evidence.
- **Evals** (control plane): invariants and recovery evals per run, offline suites per change, release gates per candidate.
- **Evidence** (under every layer): per-attempt facts in the journal; traces for timelines; metrics for SLOs; provider ledgers as the ground truth for effects.

43 • Operational runbook

When an agent step fails, the default response by signal, as the POC's matrix implements it:

Signal	Execution certainty	Side effect	Default response	Rule
validation error, invalid tool, invalid output	NOT_EXECUTED (blocked)	none	regenerate once with the error; then escalate	M03, M05
authorization denied	NOT_EXECUTED	none	abort; route to approval if the request is legitimate	M02
model 503 / 429	NOT_EXECUTED	none	retry once, then fall back to another model	M06, M07
refused connection, shed request, deadline refusal	NOT_EXECUTED	none	retry within budget	M17
documented business rejection	NOT_EXECUTED	none	escalate; do not retry	M09
2xx	EXECUTED	present	continue	M01
read timed out	UNKNOWN	none	retry	M18
write timed out, provider queryable	UNKNOWN	possible	reconcile after the deadline	M19
reconciled: found one	EXECUTED	present	continue; never resend	M13
reconciled: found none	NOT_EXECUTED	none	resend under the same operation id	M16
reconciled: found more than one	EXECUTED	duplicated	compensate if undoable, else escalate	M14, M15
reconciliation unavailable, key fresh	UNKNOWN	possible	resend under the same key	M20
write timed out, no query, no key	UNKNOWN	possible	stop and escalate ; report the effect as unknown	M21
worker crash, no intent recorded	NOT_EXECUTED	none	resume the step	M12
worker crash, intent but no result	UNKNOWN	possible	reconcile	M19
worker crash, result recorded	EXECUTED	present	continue without replay	M13
result could not be persisted after commit	EXECUTED	present	fail-stop; a fresh worker reconciles	M11
journal inconsistent	—	unknown	stop; escalate	M22
no rule matches	—	—	escalate; never guess	M99

44 • Limitations

- **Simulated providers.** They implement the documented contracts the experiment depends on (idempotency windows, enforced deadlines, status queries, commit-then-lose), and are idealised in others: reconciliation reads are immediately consistent; operation history is never

pruned; there is no partial success, no rate limiting on status queries, no clock skew between runtime and provider.

- **A fixed workflow.** The agent follows an orchestration graph; an agent that chooses its own sequence of tools needs the same machinery per call, plus trajectory evals that tolerate valid reorderings.
- **One real model in the end-to-end run.** The controlled fault scenarios use a scripted decision step by design; the end-to-end run repeats them with qwen3:8b, which made the same decisions as the stand-in on these three cases. Other models, other prompts or a higher temperature could change the decisions; the recovery semantics are designed not to depend on them, and this run shows they did not for this one. The real-model slice is sixteen cases and two 8B-class local models: a demonstration of the gate, not a benchmark of models.
- **Scenario coverage, not distribution.** 25 faults, one run each, deterministic. The results say what each runtime does under each fault; they do not say how often those faults happen, or what the duplicate rate of a real system would be.
- **No load.** Retry storms, backoff, budgets and circuit breakers were outside the experiment.
- **Escalation is a status, not a workflow.** Nothing measures what the person does with it.
- **Effectively-once, not exactly-once.** No duplicate occurred in any of the measured scenarios under the classified runtime; S12 shows the case where it cannot be ruled out, and the runtime says so instead of guessing.

45 • Final production checklist

- [] Every run has a stable correlation identity, stored durably, restored on resume.
- [] Failure classes are structured, per layer; there is no generic `agent_error`.
- [] Every tool attempt records `request_sent` and `response_received` separately.
- [] Execution certainty (`NOT_EXECUTED` / `EXECUTED` / `UNKNOWN`) is represented explicitly.
- [] Every tool has a contract: effect, idempotency + window, status query, undo, pre-execution statuses.
- [] External writes have a stable operation identity, sent as the idempotency key.
- [] Intent is persisted before dispatch; the result after the outcome.
- [] Unknown outcomes on writes trigger reconciliation, after the request's provider-enforced deadline.
- [] Authorization denials and business rejections are never retried.
- [] Validation failures are repaired or escalated, never resent, never counted as outages.
- [] Recovery is a deterministic, versioned policy; every decision records its rule.
- [] Irreducible uncertainty escalates to a person, with the evidence attached.
- [] Answers are built from confirmed state; `UNKNOWN` is reported as unknown.
- [] Trace identity survives retries, crashes and resume; resumed workers link back.
- [] Recovery actions are observable as their own facts, not only as span errors.
- [] Invariants run on every execution; recovery evals run on every change.
- [] The eval suite is mutation-tested: each check has failed at least once on purpose.
- [] Model, prompt, tool, policy and matrix changes pass offline evals before release.
- [] Telemetry contains identifiers and decisions, never secrets or personal data.
- [] Dashboards include unknown-outcome rate, reconciliation, escalations and duplicates.

46 • Conclusion

The goal is not to make agents never fail. It is to make failure **observable, classifiable, recoverable, and verifiably safe**.

A production agent should never ask only “did something fail?”. It should ask what failed, what happened before it failed, whether the external action executed, what state survived, what action is safe now, and how the recovery will be verified. In this run, the runtime that asked those questions committed 0 duplicate effects across 25 injected faults, gave 0 false answers, and was caught by its own evals every time one line of its recovery layer was broken.

“The agent failed” is not an operational signal.

References

Every source was fetched on 2026-10-07 and is quoted in `research/sources.md` with the passage it supports and what it does *not* support. The historical results of F2, T5 and P1 are read from those packages' own recorded facts.

[1] Jepsen tutorial, "Writing a client" — Jepsen project (Kyle Kingsbury), GitHub `jepsen-io/jepsen`, `doc/tutorial/03-client.md` (last changed at commit `3c72775ce5`, 2023-04-23). github.com/jepsen-io/jepsen/blob/main/doc/tutorial/03-client.md

[2] Status codes and their use in gRPC — gRPC documentation (page last modified 2024-08-21). grpc.io/docs/guides/status-codes/

[3] Retry — gRPC documentation. grpc.io/docs/guides/retry/

[4] Tools — Model Context Protocol specification, revision 2026-07-28. modelcontextprotocol.io/specification/2026-07-28/server/tools

[5] Retry Policies — Temporal documentation. docs.temporal.io/encyclopedia/retry-policies

[6] Handling errors in Step Functions workflows — AWS Step Functions Developer Guide. docs.aws.amazon.com/step-functions/latest/dg/concepts-error-handling.html

[7] RFC 9110 — HTTP Semantics, §9.2.2 Idempotent Methods — IETF (June 2022). www.rfc-editor.org/rfc/rfc9110.html#section-9.2.2

[8] The Idempotency-Key HTTP Header Field, draft-ietf-httpapi-idempotency-key-header-07 — IETF HTTPAPI WG (J. Jena, S. Dalal), 15 Oct 2025. datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/

[9] Idempotent requests — Stripe API Reference. docs.stripe.com/api/idempotent_requests

[10] Making retries safe with idempotent APIs — Amazon Builders' Library (Malcolm Featonby). aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

[11] Timeouts, retries, and backoff with jitter — Amazon Builders' Library (Marc Brooker). aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/

[12] Ensuring idempotency in Amazon EC2 API requests — Amazon EC2 Developer Guide. docs.aws.amazon.com/ec2/latest/devguide/ec2-api-idempotency.html

- [13] Handling Overload — Site Reliability Engineering (Google, O'Reilly 2016), ch. 21 (Alejandro Forero Cuervo). sre.google/sre-book/handling-overload/
- [14] Addressing Cascading Failures — Site Reliability Engineering (Google), ch. 22 (Mike Ulrich). sre.google/sre-book/addressing-cascading-failures/
- [15] Design — Message Delivery Semantics — Apache Kafka 4.2 documentation. kafka.apache.org/42/design/design/
- [16] Activity Definition — Idempotency — Temporal documentation. docs.temporal.io/activity-definition
- [17] Detecting Activity failures — Activity Heartbeat — Temporal documentation. docs.temporal.io/encyclopedia/detecting-activity-failures
- [18] Sagas — Hector Garcia-Molina and Kenneth Salem, Proc. ACM SIGMOD 1987, pp. 249–259 (Princeton University). www.cs.cornell.edu/andru/cs711/2002fa/reading/sagas.pdf
- [19] Saga design pattern — Azure Architecture Center (Microsoft Learn). learn.microsoft.com/en-us/azure/architecture/patterns/saga
- [20] Compensating Transaction pattern — Azure Architecture Center (Microsoft Learn). learn.microsoft.com/en-us/azure/architecture/patterns/compensating-transaction
- [21] Controllers — Kubernetes documentation. kubernetes.io/docs/concepts/architecture/controller/
- [22] Semantic conventions for generative client AI spans — OpenTelemetry `semantic-conventions-genai`, commit `4f85037` (2026-10-06), no tagged release. github.com/open-telemetry/semantic-conventions-genai/blob/4f85037ef86e92c510d2ef881a58f107
- [23] Semantic Conventions for GenAI agent and framework spans — OpenTelemetry `semantic-conventions-genai`, commit `4f85037` (2026-10-06). github.com/open-telemetry/semantic-conventions-genai/blob/4f85037ef86e92c510d2ef881a58f107
- [24] Semantic conventions for Generative AI events — `gen_ai.evaluation.result` — OpenTelemetry `semantic-conventions-genai`, commit `4f85037`; README for status. github.com/open-telemetry/semantic-conventions-genai/blob/4f85037ef86e92c510d2ef881a58f107
- [25] Semantic conventions for HTTP spans — HTTP request retries and redirects — OpenTelemetry semantic conventions 1.44.0 (HTTP: Stable). opentelemetry.io/docs/specs/semconv/http/http-spans/

[26] Overview — Links between spans — OpenTelemetry Specification.

opentelemetry.io/docs/specs/otel/overview/

[27] Traces — OpenTelemetry concepts (page last modified 2026-01-14).

opentelemetry.io/docs/concepts/signals/traces/

[28] Recording errors — OpenTelemetry semantic conventions 1.44.0 (status: Development).

opentelemetry.io/docs/specs/semconv/general/recording-errors/

[29] Error attributes registry (`error.type` , `error.message`) — OpenTelemetry semantic conventions

1.44.0. opentelemetry.io/docs/specs/semconv/registry/attributes/error/

[30] Semantic conventions for exceptions on spans — OpenTelemetry semantic conventions 1.44.0.

opentelemetry.io/docs/specs/semconv/exceptions/exceptions-spans/

[31] Sampling — OpenTelemetry concepts (page last modified 2025-10-16).

opentelemetry.io/docs/concepts/sampling/

[32] Trace Context — W3C Recommendation, 23 November 2021. www.w3.org/TR/trace-context/

[33] Propagation format for distributed context: Baggage — W3C Candidate Recommendation

Snapshot, 30 May 2024. www.w3.org/TR/baggage/

[34] Handling sensitive data — OpenTelemetry documentation (Security).

opentelemetry.io/docs/security/handling-sensitive-data/

[35] Logging Cheat Sheet — OWASP Cheat Sheet Series.

cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

[36] Demystifying evals for AI agents — Anthropic Engineering (Mikaela Grace, Jeremy Hadfield, Rodrigo Olivares, Jiri De Jonghe), published 9 Jan 2026.

www.anthropic.com/engineering/demystifying-evals-for-ai-agents

[37] Building effective agents — Anthropic Engineering (Erik S., Barry Zhang), published 19 Dec

2024. www.anthropic.com/engineering/building-effective-agents

[38] Evaluation best practices — OpenAI API documentation.

platform.openai.com/docs/guides/evaluation-best-practices

[39] Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena — Lianmin Zheng et al., NeurIPS 2023 Datasets and Benchmarks Track, arXiv:2306.05685v4. arxiv.org/abs/2306.05685

[40] τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains — Shunyu Yao, Noah Shinn, Pedram Razavi, Karthik Narasimhan, arXiv:2406.12045v1 (17 Jun 2024).
arxiv.org/abs/2406.12045

[41] Berkeley Function Calling Leaderboard (BFCL) — blog (Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Ion Stoica, Joseph E. Gonzalez, Tianjun Zhang, Shishir G. Patil), last updated 2024-08-19. gorilla.cs.berkeley.edu/blogs/8_berkeley_function_calling_leaderboard.html

[42] Ragas: Automated Evaluation of Retrieval Augmented Generation — Shahul Es, Jithin James, Luis Espinosa-Anke, Steven Schockaert, arXiv:2309.15217v2 (rev. 28 Apr 2025).
arxiv.org/abs/2309.15217

[43] Identifying the Risks of LM Agents with an LM-Emulated Sandbox (ToolEmu) — Yangjun Ruan et al., arXiv:2309.15817v2 (rev. 17 May 2024). arxiv.org/abs/2309.15817

[44] agentevals — LangChain, GitHub `langchain-ai/agentevals` README (last changed at commit `25cdf7c248`, 2025-09-03). github.com/langchain-ai/agentevals

[45] Service Level Objectives — Site Reliability Engineering (Google), ch. 4. sre.google/sre-book/service-level-objectives/

[46] Embracing Risk — Site Reliability Engineering (Google), ch. 3. sre.google/sre-book/embracing-risk/

[47] Alerting on SLOs — The Site Reliability Workbook (Google), ch. 5.
sre.google/workbook/alerting-on-slos/

[48] Canarying Releases — The Site Reliability Workbook (Google), ch. 16.
sre.google/workbook/canarying-releases/

AI Agents

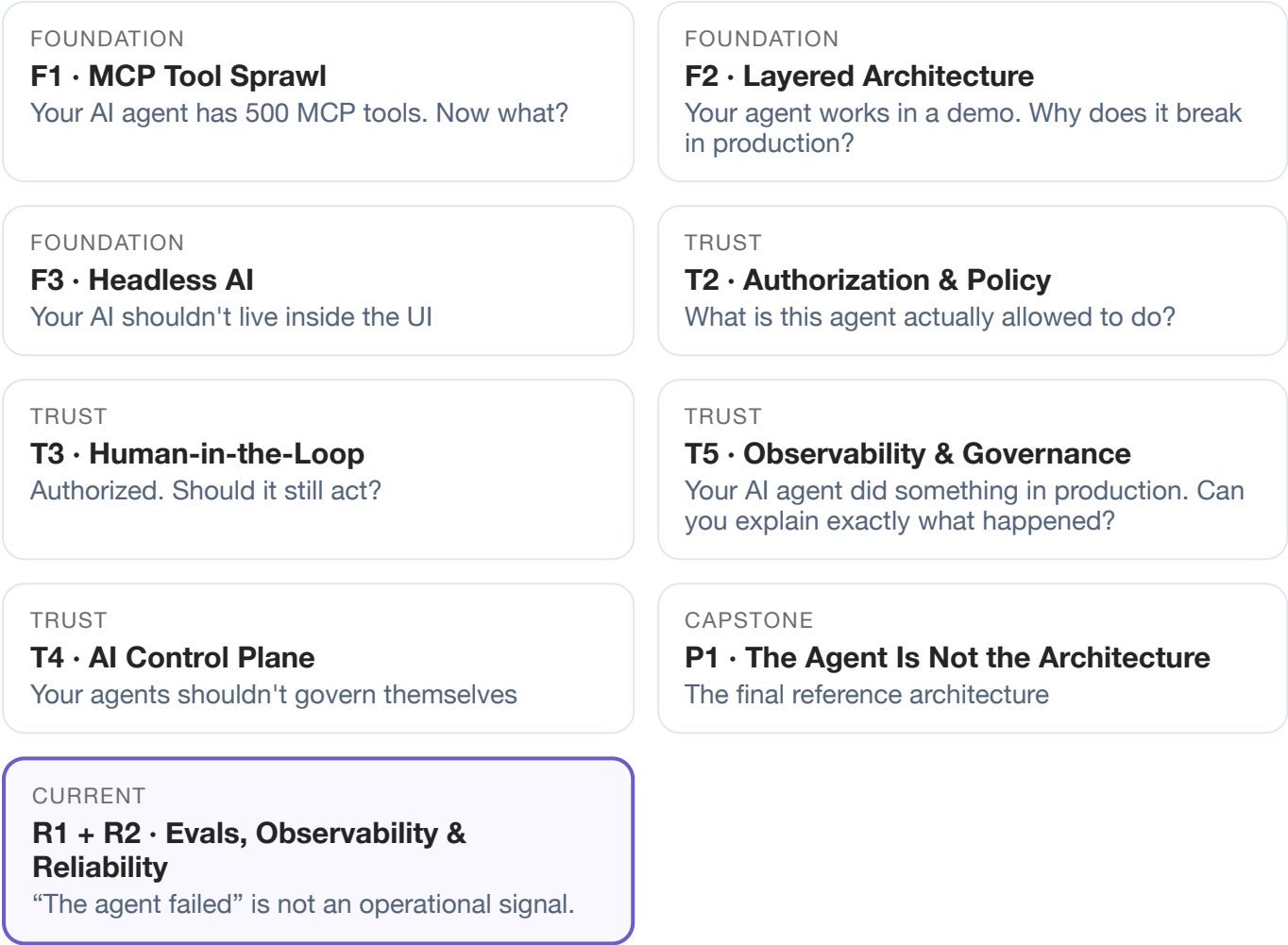
Reliability Engineering

Observability

LLM Evaluation

Distributed Systems

PRODUCTION AI ENGINEERING · THE SERIES SO FAR



The full learning map (Start Here)

R1 + R2 · Production AI Engineering

Production AI Engineering · R1 + R2. Every measured number here is substituted at build time from `recovery_poc/runs/2026-10-07-recorded/facts.json`; the providers are simulated, the worker processes, crashes, sockets and timeouts are real, and the deterministic part of the run replays byte for byte. Figures carry their provenance.

[Medium edition](#) [Evidence Check](#) [POC README](#) [Run summary](#) [Lab Console](#) [QA report](#)